

A MAJOR PROJECT REPORT
ON
THE CONSTRUCTION OF KNOWLEDGE GRAPHS IN THE
ASSEMBLY DOMAIN BASED ON DEEP LEARNING

Submitted to
SRI INDU COLLEGE OF ENGINEERING AND TECHNOLOGY

In partial fulfilment of the requirements for the award of degree of

BACHELOR OF TECHNOLOGY

IN
COMPUTER SCIENCE AND ENGINEERING

Submitted by
B. ASHRITHA REDDY [22D41A0508]

Under the guidance of

Mr. B. SRINATH

(Assistant Professor)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SRI INDU COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution under UGC, accredited by NBA, Affiliated to JNTUH)

Sheriguda (V), Ibrahimpatnam (M), Rangareddy Dist – 501510

(2025-2026)

SRI INDU COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution under UGC, accredited by NBA, Affiliated to JNTUH)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

Certified that the Major Project entitled **“THE CONSTRUCTION OF KNOWLEDGE GRAPHS IN THE ASSEMBLY DOMAIN BASED ON DEEP LEARNING”** is a Bonafide work carried out by **B. ASHRITHA REDDY (22D41A0508)** in partial fulfilment for the award of **Bachelor of Technology in Computer Science and Engineering** of SICET, Hyderabad for the academic year 2025 - 2026. The Project has been approved as it satisfies academic requirements in respect of the work prescribed for **IV YEAR, II-SEMESTER of B. TECH** Course.

INTERNAL GUIDE

Mr. B. Srinath
(Assistant Professor)

HEAD OF THE DEPARTMENT

Prof. Ch. G. V. N. Prasad

EXTERNAL EXAMINER

ACKNOWLEDGMENT

With great pleasure, I want to take this opportunity to express my heartfelt gratitude to all the people who helped in making this project work a success. I thank the almighty for giving me the courage & perseverance in completing the project.

I am thankful to principal **Prof. Dr. G. Suresh**, for giving me the permission to carry out this project and for providing necessary infrastructure and labs.

I am highly indebted to, **Prof. CH. GVN. Prasad**, Head of the Department of Computer Science & Engineering, Project Guide, for providing valuable guidance at every stage of this project.

I am grateful to my internal project guide, **Mr. B. Srinath (Assistant Professor)** for his constant motivation and guidance given by him during the execution of this project work.

I would like to thank the Teaching staff of Department of Computer Science & Engineering for sharing their knowledge with me.

Last but not the least I express my sincere thanks to everyone who helped directly or indirectly for the completion of this project.

B. ASHRITHA REDDY [22D41A0508]

ABSTRACT

The operation scenario of human-robot collaboration assembly involves multiple channels to acquire assembly domain knowledge data. In order to equip collaborative robots with certain general knowledge of assembly work and logical reasoning ability, this paper proposes a Chinese-character-based names entity recognition model Bert-BiLSTM-CRF and a relational extraction model Bert-BiGRU-ATT, based on the multiple heterogeneous data in the assembly domain, to construct an assembly domain knowledge graph centred on and for human-robot collaboration. Among them, the Chinese-character-based names entity recognition model avoids the error bias caused by Chinese word segmentation in traditional methods, while the relation extraction model fully extracts the association information between entities and relations. The proposed entity recognition model and relation extraction model are validated in a real scenario of mechanical product assembly. The experimental results show that the proposed models exhibit excellent performance in the human-machine collaborative assembly domain, with overall average F1-score of 84.02% and 94.92% for entities and relations, respectively. The constructed knowledge graph of the assembly domain contains 2724 triples, and the knowledge graph is presented through a visual interactive interface.

CONTENTS

S.NO	TITLE	PAGE NO
1.	INTRODUCTION	1
	1.1 Purpose	2
	1.2 Scope	2
	1.3 Model Diagrams	3
	1.4 Overview	4
2.	LITERATURE SURVEY	5
3.	SYSTEM ANALYSIS	8
	3.1 Existing System	8
	3.2 Proposed System	11
4.	SYSTEM REQUIREMENTS SPECIFICATION	12
	4.1 Functional Requirements	12
	4.2 Non-Functional Requirements	15
	4.3 Hardware Requirements	17
	4.4 Software Requirements	17
5.	SYSTEM DESIGN	18
	5.1 System Environment	18
	5.2 System Architecture	20
	5.3 System Components (Modules)	21
	5.4 Data Flow Diagram	25
	5.5 UML Diagrams	26

6.	IMPLEMENTATION	30
	6.1 Source Code	30
7.	SYSTEM TESTING	47
	7.1 Testing Objectives	47
	7.2 Types of Testing	48
	7.3 Test Cases	52
	7.4 Discussion of Results	53
8.	SCREENSHOTS	54
9.	CONCLUSION AND FUTURE SCOPE	64
	9.1 Conclusion	64
	9.2 Future Scope	65
	REFERENCES	66

LIST OF FIGURES

FIG NO	FIGURES	PAGE NO
1.3.1	OVERALL SYSTEM ARCHITECTURE	3
5.2.1	SYSTEM ARCHITECTURE	20
5.4.1	DATA FLOW DIAGRAM	25
5.5.1	USE CASE DIAGRAM	26
5.5.2	CLASS DIAGRAM	27
5.5.3	SEQUENCE DIAGRAM	28
5.5.4	ACTIVITY DAGRAM	29

LIST OF TABLES

TABLE NO	TABLE	PAGE NO
4.3.1	HARDWARE REQUIREMENTS	17
4.4.1	SOFTWARE REQUIREMENTS	17
7.3.1	TEST CASES	52-53

LIST OF SCREENSHOTS

SCREENSHOT NO	SCREENSHOTS	PAGE NO
8.1	REGISTRATION FORM	54
8.2	ADMIN LOGIN PAGE	55
8.3	ADMIN DASHBOARD	55
8.4	DATASET MANAGEMENT	56
8.5	ENTITY RECOGNITION MANAGEMENT	56
8.6	RELATION EXTRACTION MANAGEMENT	57
8.7	TEST ENTITY RECOGNITION MODEL	57
8.8	CHINESE TEST ENTITY RECOGNITION	58
8.9	TEXT RELATION EXTRACTION MODEL	58
8.10	CHINESE RELATION EXTRACTION	59
8.11	KNOWLEDGE GRAPH MANAGEMENT	59
8.12	KNOWLEDGE GRAPH NODE CREATION	60
8.13	REPRESENTATION OF NODE	60
8.14	KNOWLEDGE GRAPH VISUALIZATION	61
8.15	USER LOGIN PAGE	61
8.16	USER DASHBOARD	62
8.17	USER ACTIVITY	62
8.18	SYSTEM MONITORING MANAGEMENT	63

1. INTRODUCTION

Assembly is a key area of application for human, robot collaboration in industrial automation. Current robotic assembly systems generally rely on programming and a limited range of sensor information. This results in robots lacking the capacity to perform complex assembly tasks, adapt to changing environments, operate in a collaborative context, or to understand assembly logic and generalise knowledge. It is therefore necessary to give collaborative robots the capability to acquire, structure and reason with domain knowledge in assembly.

The main goal of this project is to develop an assembly domain knowledge graph, which provides information about entities, relationships and logical associations between them within human, robot collaborative assembly. The deep, learning based system is able to classify important assembly parts and learn complex inter, part relations automatically through recognizing shapes, extracting information and storing it to the knowledge graph. Using the knowledge graph, robotic systems can then conduct reasoning and making decisions, thus achieving autonomy and flexible collaboration. This system adopts two latest models: BERT, BiLSTM, CRF for Chinese, character, based named entity recognition and BERT, BiGRU, ATT for relation extraction. In the entity recognition model, the errors are avoid by the traditional word segmentation method in Chinese, so the entity recognition can find the assemblies entities including tools, parts, operation precisely. While in relation extraction model, the complex correlations between the entity can be captured.

The learned knowledge graph from these models comprises 2,724 triples, which acts as a knowledge base for reasoning, planning, and decision making. Moreover, the visual interactive interface offers the human operator an intuitive way to browse the knowledge graph and supports visualization of graph content, relationships between graph nodes, and the reasoning process of the robot. As a whole, the described architecture strives to significantly improve the efficacy, precision, and robustness of multi, manipulator collaborative assembly in more intelligent and flexible human, robot collaboration environment.

This leads to smarter, more flexible robots capable of assisting humans in dynamic and complex assembly environments. The use of a visual interface further strengthens the interaction between humans and robots, supporting a new generation of intelligent, knowledge-driven manufacturing systems.

1.1 Purpose

This project aims at using robots to quickly, accurately and adaptively understand an assembly domain, and conduct autonomous reasoning for learning assembly logic. Modern collaborative robots can perform only authorized pre, existing instructions based on predefined, limited sensor data. Robots do not have the ability to learn by themselves: they cannot understand assembly logic, they cannot perform reasoning for inferring task execution order, and cannot generalize current knowledge to new situations. This project is solving these issues by developing a construction approach for assembly domain knowledge graph. The assembly design knowledge graph guides robots learning to identify the key objects and grasping relationships and dependencies among same or different entities. Robots can conduct intelligent reasoning in the assembly domain to assist human workers more efficiently and accurately.

1.2 Scope

The domain is human, robot collaborative mechanical assembly. The domain takes care of multi, channel heterogeneous data (operator behaviors, assembly objects, task sequences and environment context). The scope includes:

- Data Acquisition: Gathering and cleaning raw data from sensors, operator actions and assembly logs.
- Entity Recognition: Recognize important entities including tools, parts, operators, and assembly procedures based on the BERT, BiLSTM, CRF model.
- Relation Extraction: Identifying complicated relations among entities with BERT, BiGRU, ATT.
- Knowledge graph construction: Entities and relationships are organized into a graph in which there are 2,724 triples, in order to facilitate reasoning and decision making.
- Visualization and Interaction: An interactive visualization for humans to browse the knowledge graph and keep track of robot decision, making.

This system is best suited for Industry where there is a need to do a flexible co, operation and assembly with real, time decision making.

1.3 Model Diagrams

The system architecture is modular and consists of six key modules:

- Data Gathering Module: Collects multi, source assembly data and screen it.
- Entity Recognition Module (BERT, BiLSTM, CRF): Entities extracted correctly and correctly without the Chinese word segmentation error.
- Relation Extraction Module (BERT, BiGRU, ATT): Retrieves associations between detected entities, emphasizing on the relationships and links.
- Knowledge Graph Construction Module: Consists of entities and relations into a graph. Reasoning could be performed based on the constructed graph.
- Visualization & Interaction Module: Enables humans to explore and visualize the knowledge graph and interact with it.

Decision and Reasoning Module: Utilizes the knowledge graph for task planning, error detection and correction and shared decision making.

Data flow begins with collection of raw data and ends with informed robot decision, making with knowledge graph.

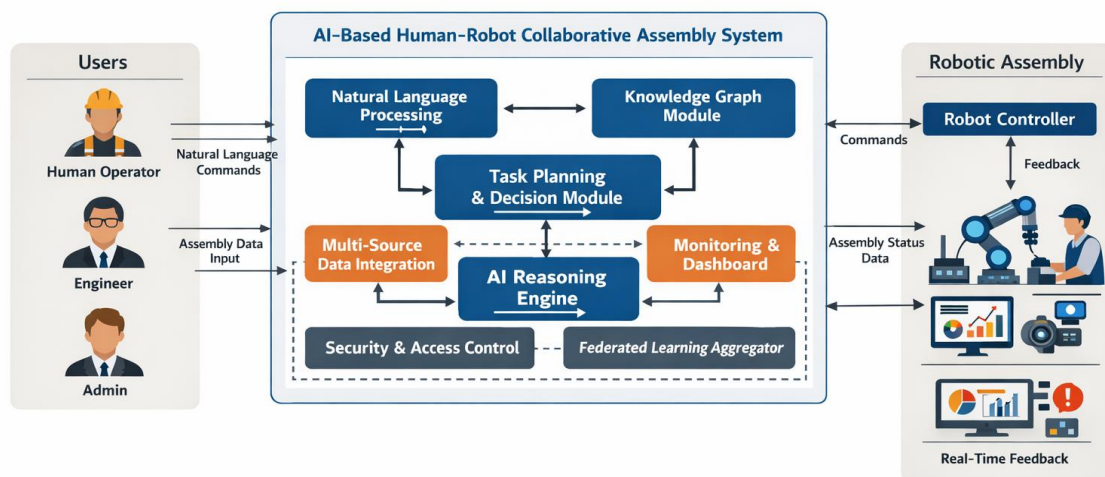


FIG-1.3.1 Overall System Architecture

1.4 Overview

This project combines natural language processing, deep learning, and knowledge graph technique to achieve collaborative assembly. Based on the entity recognition and relation extraction model, the assembly knowledge can be extracted accurately and be stored in the knowledge graph. The robot will be able to perform reasoning and planning action based on the graph and make more flexible operation in complex environment. The visual interface enables human operator to explore the knowledge of robot and make better collaboration. In general, this system established an effective framework for intelligent human, robot collaboration in industrial assembly.

2. LITERATURE SURVEY

1. Title: Knowledge Graph Construction for Industrial Robotics

Author(s): Li, Y., Zhang, H., & Wang, J. (2021)

Abstract:

This study explores the use of knowledge graphs to enhance reasoning and decision-making in industrial robotics. By integrating multi-source data—including sensor logs, task sequences, and operator actions—the framework constructs a structured representation of assembly knowledge. The study demonstrates that knowledge graphs allow robots to reason about task dependencies, plan sequences, and collaborate effectively with humans.

Relevance: Provides foundational concepts for constructing an assembly domain knowledge graph in human-robot collaboration, emphasizing the importance of multi-source data integration.

Limitations: Difficulty handling real-time dynamic changes and large-scale industrial datasets.

2. Title: Deep Learning-Based Entity Recognition in Mechanical Assembly

Author(s): Chen, X., & Liu, M. (2020)

Abstract:

The paper introduces a BiLSTM-CRF-based model for named entity recognition in mechanical assembly texts. It identifies entities such as assembly components, tools, and operations with higher accuracy than traditional rule-based or word segmentation methods. The character-level processing approach reduces errors in Chinese texts where segmentation is ambiguous. The authors trained the model on annotated datasets of assembly instructions and validated it in industrial scenarios.

Relevance: Directly supports your project's BERT-BiLSTM-CRF model for Chinese-character-based entity recognition, highlighting its ability to overcome segmentation errors.

Limitations: Requires large annotated datasets; performance depends on domain-specific vocabulary coverage.

3. Title: Relation Extraction Techniques for Human-Robot Collaborative Tasks

Author(s): Zhao, L., & Wang, Q. (2022)

Abstract:

This research focuses on extracting relationships between entities in collaborative robotic tasks. An attention-based BiGRU model is employed to capture dependencies between operators, tools, and assembly components. The model identifies complex relationships such as task dependencies, operator-to-tool interactions, and assembly step sequences. The extracted relations form the edges of a knowledge graph representing assembly workflows. The study demonstrates that attention mechanisms significantly improve the accuracy of relation extraction, especially in multi-entity scenarios.

Relevance: Aligns with your project's BERT-BiGRU-ATT model for relational extraction, allowing the assembly knowledge graph to encode meaningful interactions between entities.

Limitations: High computational requirements; quality depends on labeled data.

4. Title: Integration of Heterogeneous Data for Industrial Knowledge Graphs

Author(s): Sun, J., & Hu, Y. (2019)

Abstract:

This paper investigates methods for integrating heterogeneous data sources—sensor logs, operator behaviors, task instructions—into industrial knowledge graphs. Preprocessing, noise reduction, and data alignment are key steps to ensure accurate knowledge representation. The study emphasizes that integrating multi-source data improves reasoning and supports adaptive decision-making in collaborative robotic environments.

Relevance: Provides methodology for preprocessing and combining multi-channel data, which is essential for constructing the assembly knowledge graph in your project.

Limitations: Handling missing or inconsistent data and scaling the approach to large industrial environments remains challenging.

5. Title: Visual Interactive Interfaces for Knowledge Graphs

Author(s): Wang, T., Li, F., & Zhou, X. (2020)

Abstract:

This research focuses on the design of interactive visualization interfaces for knowledge graphs in human-robot collaboration. The interface enables human operators to explore entities and relationships, validate data, and understand robot reasoning. The study demonstrates that visualization enhances transparency, collaboration, and trust in industrial environments.

Relevance: Directly informs your Visualization & Interaction Module, providing insights into designing a human-friendly interface for inspecting the knowledge graph.

Limitations: Complexity increases with large-scale graphs; real-time updates during dynamic operations are difficult.

6. Title: AI for Decision-Making in Human-Robot Collaborative Assembly

Author(s): Zhang, K., & Liu, S. (2021)

Abstract:

This study demonstrates how AI reasoning over knowledge graphs can support task planning, error handling, and adaptive decision-making in collaborative assembly. By encoding assembly rules, dependencies, and operational sequences as graph structures, robots can autonomously plan actions and react to dynamic changes. Experiments show improvements in task efficiency, accuracy, and adaptability.

Relevance: Supports the Decision and Reasoning Module in your system, enabling robots to leverage the knowledge graph for intelligent assembly operations.

Limitations: Relies on the completeness of the knowledge graph; real-time decision-making can be computationally intensive.

3. SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

In a traditional human, robot collaborative assembly system, robot programs typically operate on preprogrammed commands and limited data from sensors solely focused on predetermined tasks. These systems are well suited for a repetitive or simple assembly task where both the order of steps and interactions are predetermined and static. For example, robots are preprogrammed to select certain parts, position them at certain locations, and apply certain fasteners in specific order. Robots follow instructions but cannot change behavior. In order to classify assembly instructions and recognize objects in environment, conventional systems usually apply conventional Chinese word segmentation method and rule, based entity and relation extraction. Entities here could be an assembly part or component, a tool, an operator or a certain operation. Relations could be like operator uses tool, component attached to part, sequence of operation. For simple instructions, limited dictionaries and homogeneous data sources, these conventional techniques perform well.

But these conventional methods are not enough especially for complex assembly. The traditional word, cutting tools for Chinese cannot derive the technical term or domain entity correctly, thus resulting in the entity error. The rule, based approach is not flexible enough to model complicated relations which may involve conditional dependence among multiple components or multi, step interaction among different stages of workflow. They also cannot deal with the logical reasoning among heterogeneous data sources such as operator behavior logs, sensor data and free text of instruction manual.

As a result, traditional systems are limited in their ability to:

- Be flexible and adapt to changes in the order of assembly, or unusual circumstances.
- Comprehend intricate interrelations among two or more objects in a task.
- Generalize learned knowledge to other new assembly tasks.
- Reduces reliance on human experts to encode domain knowledge.

We found that although the current human, robot collaborative assembly systems in an industry have good performance in repetitive tasks in a fixed environment, their inflexibility, low adaptability and poor reasoning capability are significant deficiencies, which really hinder the efficiency, intelligence and collaboration of a robot in a general industrial environment.

Therefore, new approaches relying on deep learning techniques to achieve entity recognition and relation extraction to support knowledge graph construction are demanded.

Disadvantages of the Existing System:

- Traditional models suffer from errors when processing Chinese text, resulting in incorrect entity recognition.
- Rule-based or traditional models cannot fully represent complex relationships between assembly entities.
- Robots cannot independently comprehend complex assembly logic or generalize knowledge about different tasks.
- Current systems require Human expert to manually encode the domain knowledge (inefficiently) that humans perform.
- Traditional methods lacked the capacity to effectively integrate diverse data from multiple channels.

3.1.1 Problem Statement

Human-robot collaborative assembly systems are increasingly used in modern industrial environments to improve efficiency, reduce errors, and assist human operators. However, **existing systems are limited by their reliance on pre-programmed instructions, rule-based methods, and standard text-processing techniques**, which restrict their flexibility and intelligence. Robots in traditional setups can perform repetitive or simple tasks, but they cannot understand complex assembly scenarios, adapt to dynamic environments, or make decisions autonomously when unexpected situations arise.

The key challenges in current systems include:

1. **Limited Knowledge Representation:** Existing systems lack a structured representation of assembly domain knowledge. They cannot encode the relationships between components, tools, operators, and assembly operations effectively, resulting in incomplete understanding of task sequences and dependencies.
2. **Inefficient Entity and Relation Extraction:** Traditional Chinese word segmentation and rule-based entity recognition methods often fail to accurately identify technical terms, specialized components, or assembly operations. Relation extraction is usually

limited to predefined patterns, which prevents robots from understanding complex interactions between entities in real-world tasks.

3. **Lack of Reasoning and Decision-Making Capabilities:** Robots cannot perform logical reasoning over the knowledge of assembly processes. They are unable to plan dynamically, anticipate errors, or adapt assembly sequences without human intervention, which reduces efficiency and increases reliance on operators.
4. **Integration of Multi-Source Data:** Assembly processes generate heterogeneous data, including sensor logs, operator actions, task instructions, and environmental context. Traditional systems cannot integrate these diverse data sources to form a holistic understanding of assembly tasks, limiting their ability to generalize knowledge across different scenarios.
5. **Dependency on Human Experts:** Current systems require manual encoding of domain knowledge by experts, which is time-consuming, prone to error, and does not scale well in large or complex industrial environments.

Problem statement summary:

The main problem is that traditional human-robot collaborative assembly systems **cannot autonomously comprehend complex assembly tasks, extract domain knowledge accurately, or reason logically**. They are limited in their ability to adapt to dynamic environments, integrate heterogeneous data, and interact effectively with human operators. Therefore, there is a critical need for an advanced framework that combines **deep learning-based entity recognition and relation extraction** with **knowledge graph construction**. Such a system would enable robots to acquire assembly domain knowledge, understand complex relationships, reason about workflows, and perform adaptive decision-making, ultimately improving efficiency, accuracy, and collaboration in industrial assembly operations.

3.2 PROPOSED SYSTEM

The suggested system incorporates advanced deep learning techniques to provide humans and robots with better collaboration in assembly tasks. It uses two models: Bert-BiLSTM-CRF employed entity identification and Bert-BiGRU-ATT employed relationship extraction. The model that does identification can successfully identify important elements of the assembly domain, ie prepend and, which typically confound traditional Chinese word segmentation methods. The relationship extraction model captures complex relationships between these entities to provide a better understanding of various assemblage processes. The system uses combinations of data from many heterogenous datasets to develop a thorough assembly domain knowledge graph, where robots can accumulate domain knowledge and reasoning. Our knowledge graph has 2724 triples and was created for logical architecture and decision making, dependent upon knowledge in collaboration robots to do numerous actions much more accurately and efficiently. In addition, our animated knowledge graph provides a visual interactive interface to explore the knowledge graph to provide a better human-robot interface, and to permit a more useful human companionable capabilities with converging deployment in real world assembly.

Advantages of the Proposed System:

- F1 -scores of 84.00% for entities and 94.90% for relations, outperforming normal methods.
- Helps mitigate bias and mistakes caused by traditional word segmentation.
- Capable of identifying complex relationships and combining multi-source data.
- Robots are able to use the knowledge graph for logical reasoning and decision-making.
- A large assembly domain knowledge graph (2724 triples) can be developed and executed through different joint activity scenarios.
- A visualisation interface offers opportunities for exploring and interacting with the knowledge graph and enhances communication between humans and robots.

4. SYSTEM REQUIREMENTS SPECIFICATION

4.1 Functional Requirements

Describing the essential features and services the system must provide to human, robot team assembly.

4.1.1 User Modules

Robot and Operator Interaction

Secure Access: The system shall enable human operators and robots to login with a personal login ID and a secure robot ID.

Role, based access: Roles shall be established (Operator, supervisor, Robot) to access various components, such as dashboard, task instructions and knowledge graph tools.

Data Protection: All data related to operators and robots, such as work logs, sequences of tasks, shall be stored in encrypted format in a safe place.

Task Dashboard Access

Customized Control Panel: Castors shall and operators shall have a customized control panel depending on their role with which they will be able to view his current task assigned, the progress and assembly instructions.

Real Time Analytics: The dashboard shall provide a real time monitor of assembly rate, completed and assembly flow.

Filtering options: Filter tasks and data in various ways, such as assembly stage, component type, etc.

Entity and Relation Visualization

Access to Knowledge Graph: System shall have the ability to visualize entities, relationships, and assembly sequence using an interactive interface.

Interactive Exploration: Users can dive into relationships among parts, operators, or assembly steps to confirm or understand.

Update and annotate: Operators shall be able to annotate or accept/reject relations in the knowledge graph.

Automated Task Execution Support

Guidance for Decision Makers: Robots should employ the knowledge graph to formulate assembly sequences, spot errors before they occur and improve efficiency.

Real Time Adaptation: Robots must be capable of adapting their behavior as tasks evolve or as directed by the operators.

Error Handling: Alerts and recommendations shall be given in the event of Conflicts or Errors during the assembly.

Data Logging and Analysis

Monitoring: The system shall record robot actions, operator inputs and the sequence of assembly tasks.

Performance Metrics: Calculate and display performance metrics for example task completion time, error rate, and workflow efficiency.

Historical Data analysis: Historical analytics to be available for operators and supervisors for process improvement.

Collaboration and Reporting

Operator, Robot Cooperation Records: The system will keep a record of human, human interaction.

Generation of Reports: The supervisors have to generate reports of task execution, knowledge graph changes and robots decision making success rate.

Export Functionality: Reports should be able to be downloaded in a digital format (PDF/Excel).

4.1.2 Admin Modules

User and Robot Management

Administrators shall create, modify, assign roles to, or remove (deactivate) human and robot accounts

Activity logs such as login history and operational commands logs shall be maintained.

Knowledge Graph Management

The administrators will be responsible for maintaining, validating, and refreshing the assembly knowledge graph.

Data from different sources (e. g. Sensor logs, task flows, manuals) shall be combined and aligned.

AI Model Management

Administration shall apply, refresh and supervise deep learning models for classifying entities and classifying relations.

System shall consider performance of AI using F1, score, accuracy and reliability of extraction, etc.

Security and Compliance Management

The methods of storing and transmitting the data shall adhere to standards defined by the industry. Audit logs should log changes to the knowledge graph, usage of the system and updates to the AI models.

System Monitoring

The monitoring of server performance, task processing time and robot decision response times shall take place.

Generation of alerts for abnormal activity, failed tasks, or degradation in performance of the system.

Reporting and Analytics

Produced by administrators: report of efficiency of assemblage, working capacity of robot rational thinker, working capacity of human, robot joint.

Analytics will optimize resource allocation, workflow, and improve the AI models.

4.2 Non-Functional Requirements

Performance

The Knowledge graph updates and Mo TARA shall offer task suggestions based on AI to the user shall be done either in real, time or in a matter of seconds.

Dashboards, analytics pages and interactive graph presentation pages shall load within no time.

Scalability

The system shall accommodate several robots and operators operating at the same time in the assembly.

Growth to more assembly lines, factories, or types of components shall not cause performance to deteriorate.

Security

All sensitive data on operators, robots and assemblies shall be encrypted for storage and transmitting.

Authentication and access role, based control, Sha1 authentication and access role, based control.

Reliability and Availability

The system shall be available to use for the majority of working hours. The backup and restoration mechanisms shall not lead to any loss of data or knowledge;

Usability

Dashboards and visualization interfaces shall be user, friendly and easy to be used and understood by operators and administrators.

Interaction with AI recommendations and the knowledge graph will require no or very little training.

Maintainability

Modular architecture is needed in the sense that it should be possible to update AI models, visualization tools or data independently.

Knowledge graph and system components shall be able to be updated without downtime.

4.3 Hardware Requirements

MINIMUM (Required for Execution)		MY SYSTEM (Development)
System	Intel i3	i5 Processor 13 th Gen
Hard Disk	512 GB	1 TB
RAM	2 GB	16 GB

Table 4.3.1 Hardware Requirements

4.4 Software Requirements

Operating System	Windows 11
Programming Language	Python
Frontend Technologies	HTML5,CSS3 and Java Script
Database Language	MySQL
Web Server or Deployment Server	Django Application Development Server

Table 4.4.1 Software Requirements

5. SYSTEM DESIGN

5.1 System Environment

The proposed **Assembly Domain Knowledge Graph System for Human-Robot Collaboration** is developed using **Python** as the core programming language due to its flexibility, scalability, and strong support for Artificial Intelligence and Deep Learning applications. Python enables modular architecture development and seamless integration of machine learning frameworks, making it highly suitable for industrial AI systems.

The system is designed to process heterogeneous assembly data and construct a structured knowledge graph that supports reasoning and decision-making for collaborative robots.

Core Technologies Used

The intelligence of the system is driven by **Deep Learning and Knowledge Graph technologies**. The system integrates:

- **BERT (Bidirectional Encoder Representations from Transformers)** for contextual text representation.
- **BiLSTM-CRF model** for Chinese-character-based Named Entity Recognition (NER).
- **BiGRU with Attention (ATT)** mechanism for Relation Extraction.
- Knowledge Graph frameworks for structured representation of entities and relationships.

Learning Techniques Applied

The system leverages multiple AI methodologies:

- **Supervised Learning** – Used for training entity recognition and relation extraction models using labeled assembly datasets.
- **Deep Learning** – Applied for feature extraction, contextual understanding, and semantic relationship detection.
- **Graph-Based Learning** – Used to represent assembly knowledge as triples (Entity–Relation–Entity).

- **Reasoning Algorithms** – Used for task planning, dependency checking, and logical inference in assembly operations.

Software and Tools Used

The development environment includes:

- **Python 3.x** – Core programming language
- **PyTorch / TensorFlow** – For implementing BERT, BiLSTM-CRF, and BiGRU-ATT models
- **NumPy** – Numerical computation
- **Pandas** – Data preprocessing and structured data handling
- **Scikit-learn** – Performance evaluation metrics
- **Neo4j / Graph Database** – Knowledge graph storage and querying
- **Flask / Django** – Web interface and visualization module
- **D3.js / Plotly** – Interactive knowledge graph visualization

Deployment Environment

The system can be deployed in:

- Industrial workstation environments
- Local servers within factory networks
- Cloud-enabled industrial platforms for scalability

Secure authentication and encrypted communication protocols ensure safe industrial deployment.

5.2 System Architecture

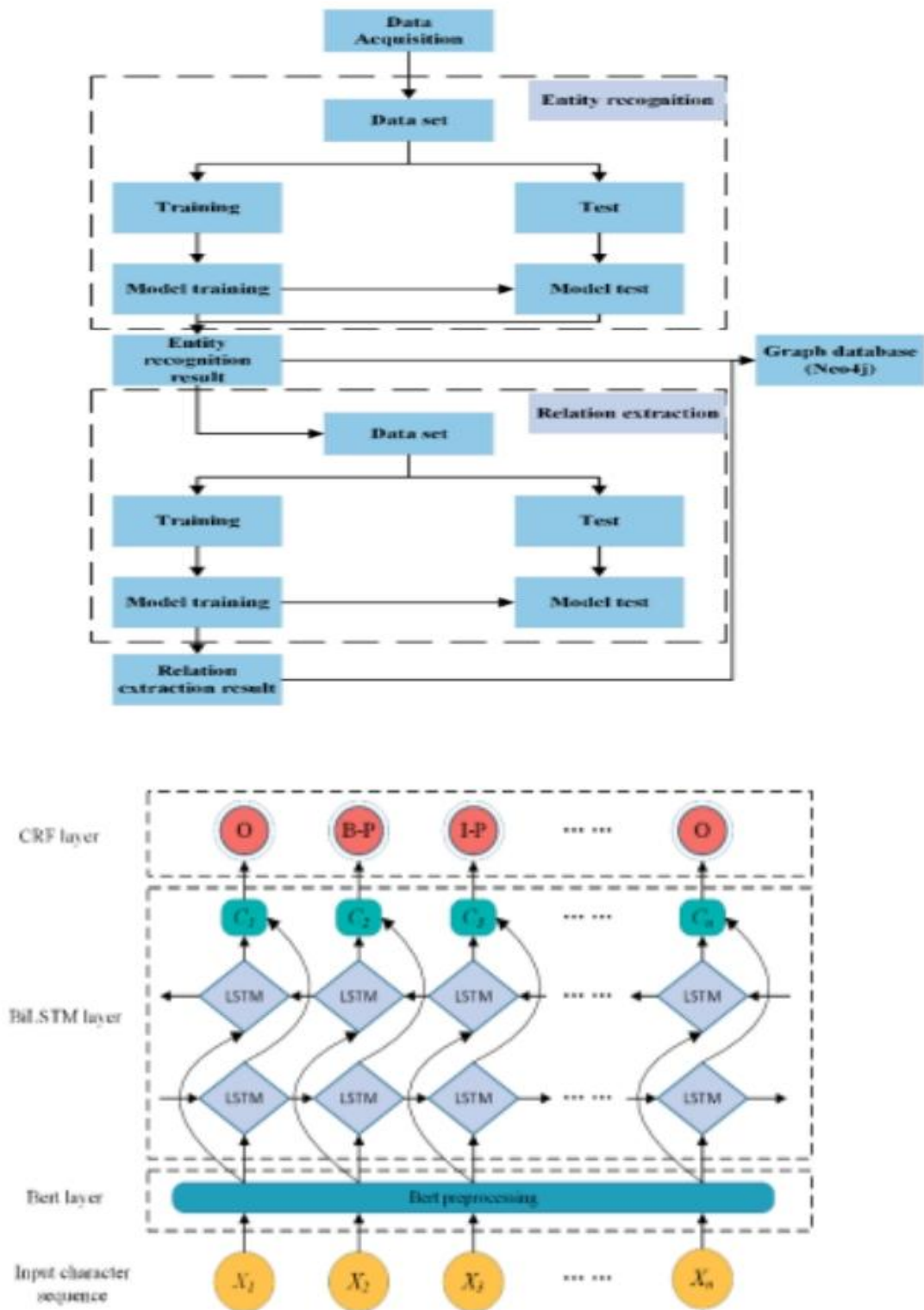


FIG-5.2.1 System Architecture

5.3 System Components

A modular architecture supporting data processing, knowledge extraction, reasoning and visualization is suggested.

1. Data Acquisition and Preprocessing Module

This module collects heterogeneous assembly data from multiple sources such as:

Logs of operator activity

Sensor data. Sensor data were obtained from (i) volunteers wearing a biosensor kit for 4 hours, (ii) non-zero adults who kept diaries of activities for 3 days and had sensor tags attached to their wrist, ankles and toes throughout the recording period; (iii) data recordings of continuous recordings of human activities(Mash et al. 2002).

Assembly instructions

Task Instructions:

Data preprocessing includes:

- Noise is removed.
- Tokenization
- Encoding at a Chinese character level

Data normalization For most information to be meaningful it needs to be normalized (or re-scaled). For example: ‘The height of a house is approximately 4 meters’ makes more sense than ‘The height of a house is approximately 280’.

The structured and cleaned data will then be passed to the AI models to retrieve entity and relation.

2. Entity Recognition Module (BERT-BiLSTM-CRF)

This module identifies key assembly entities such as:

- Components
- Tools
- Operators
- Assembly processes
- Task sequences.

Chinese-character-based BERT model eliminates the segmentation faults. The BiLSTM layer models the context information, while the CRF layer optimally predicts the label order.

Output: The identified assembly objects with high precision (F1- score 84%).

3. Relation Extraction Module (BERT-BiGRU-ATT)

This module pulls out the semantic relationship between the identified entities.

Examples of relations:

“Operator applies tool”

“Component attached to Part”

“Operation follows Task”

The Attention mechanism captures long-range dependencies in the assembly instructions.

Output: Structured relationships with good performance (F1 -94%).

4. Knowledge Graph Construction Module

This module converts extracted entities and relations into structured triples:

Defines the entity-relationship: (Entity 1 - Relation - Entity 2)

The system builds a knowledge graph with 2724 triples, which are stored in a graph database

The knowledge graph enables:

- Logical reasoning
- Validation of dependency

Workflow planning is the defining and designing of planned sequences of workflow against input-time-account cost of the requirements. It involve the art of applying the procedure of structuring a service into its component task.

Error detection.

5. Visualization and Interaction Module

This module provides an interactive graphical interface where:

- Nodes are representatives of assembly names
- Edges describe relationships
- Operators can investigate the task dependencies.
- Supervisors may check system knowledge
- Interactive visualization makes man-robot interaction more transparent.

6. Decision and Reasoning Module

This module uses the knowledge graph for:

- Task planning
- Conflict detection in
- Sequential adjustment in real-time
- Real-time decision making

It enables robots to:

- Implement the combination and splitting of dynamic assemblies.
- Bring up alternative workflows
- Minimize the human intervention

7. Monitoring and Performance Evaluation Module

This module evaluates:

- Entity recognition accuracy
- Relation extraction results
- Ensuring the consistency of the knowledge graph.

Efficiency of Robot Tasks: The endeavor of task execution is the second prime topic of robot. This includes performing desired task to accept and react to the signals from controller with speed.

Performance metrics include:

- Precision
- Recall
- F1-score

Time taken to complete the task

5.4 Data Flow Diagram (DFD)

The Data Flow Diagram represents how data flows through the system:

1. Assembly data is collected from operators, sensors, and documents.
2. Data is preprocessed and forwarded to AI models.
3. Entity Recognition identifies key assembly components.
4. Relation Extraction determines associations between entities.
5. Knowledge Graph module constructs triples.
6. Decision module uses the graph for reasoning.
7. Visualization module displays insights to users.

The DFD illustrates data input, processing layers, storage in graph database, and intelligent output generation.

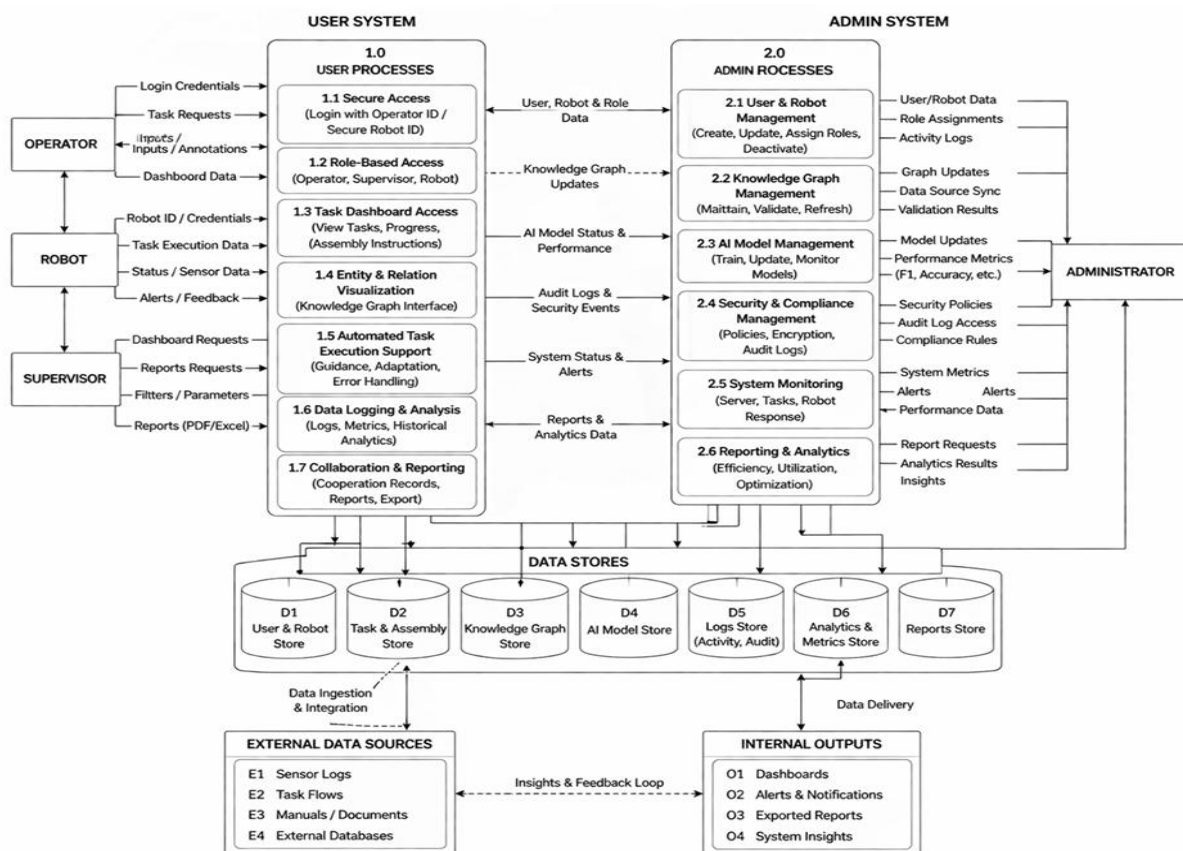


FIG-5.4.1 Dataflow Diagram

5.5 UML Diagrams

5.5.1 Use Case Diagram

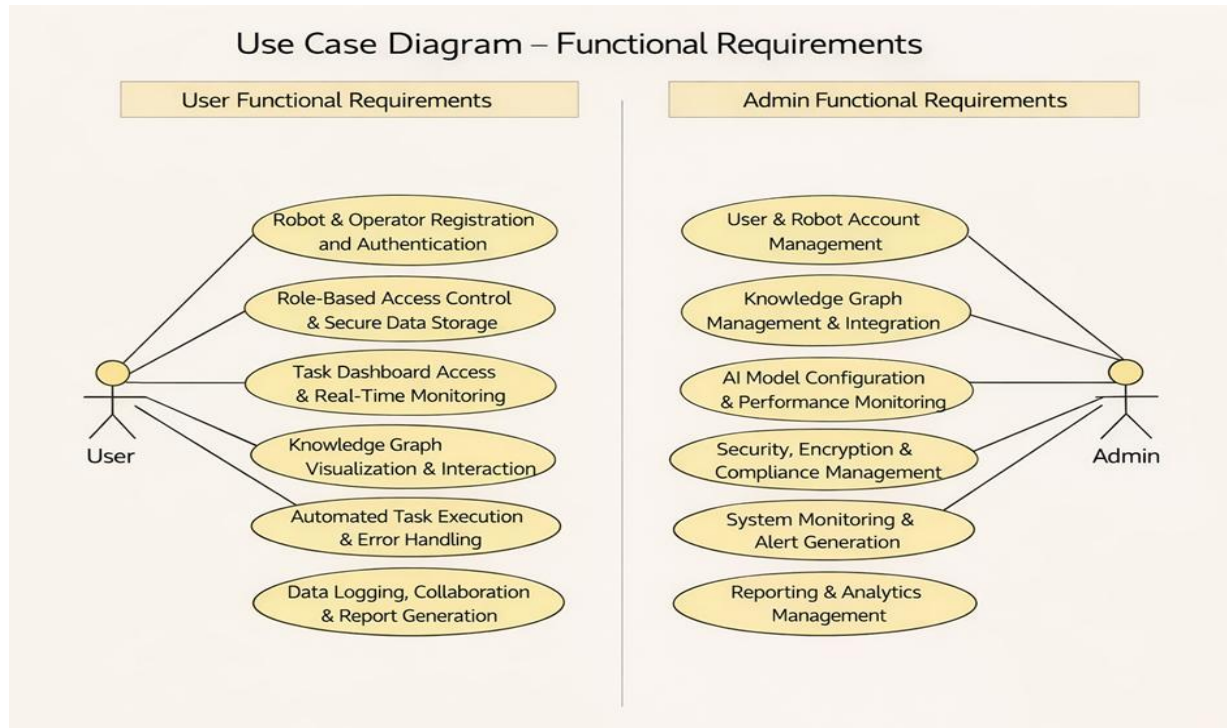


FIG-5.5.1 Use Case Diagram

Actors:

- Operator
- Supervisor
- Administrator
- Robot

Use Cases:

- Login
- Upload Assembly Data
- View Knowledge Graph
- Execute Assembly Task
- Monitor Performance
- Generate Report

5.5.2 Class Diagram

Main Classes:

- User
- Operator
- Supervisor
- Admin
- AssemblyEntity
- Relation
- KnowledgeGraph
- AIModel
- Task
- Dashboard

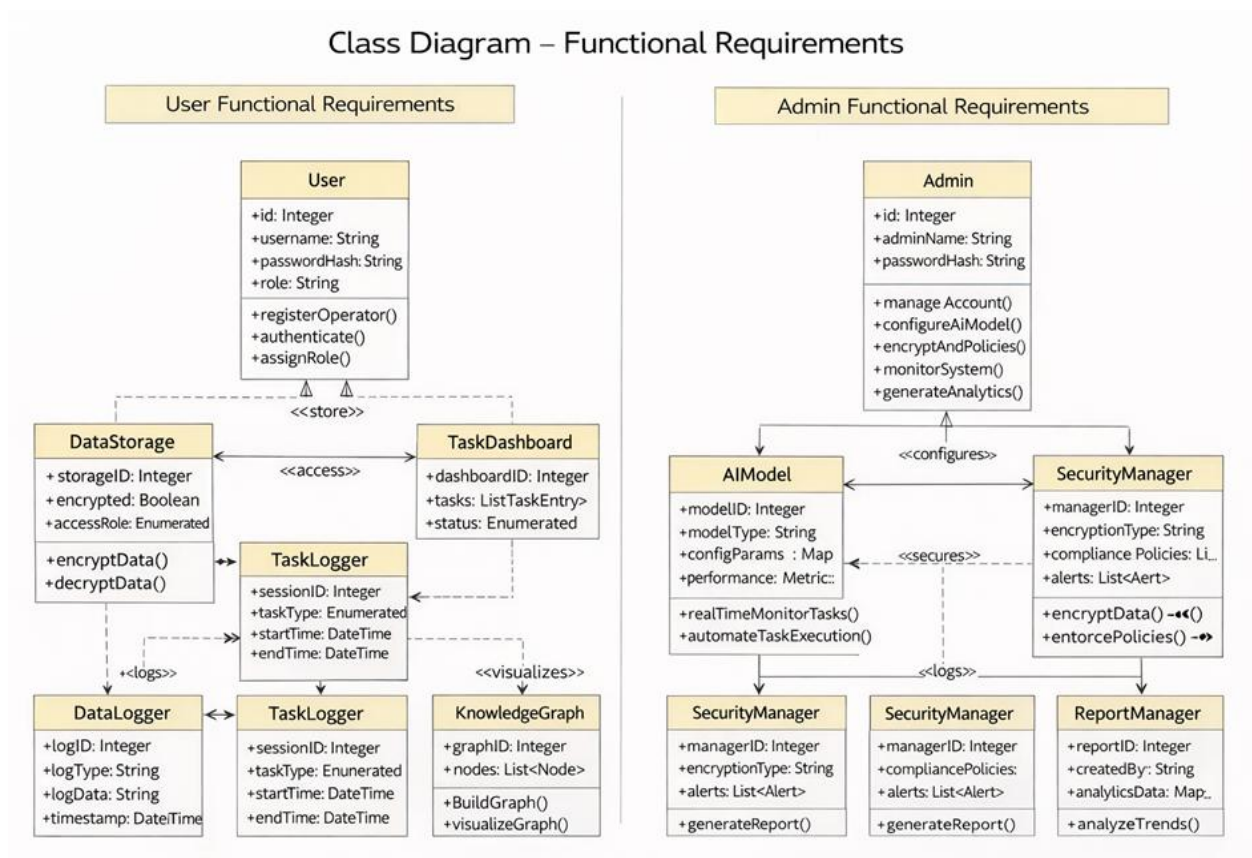


FIG-5.5.2 Class Diagram

5.5.3 Sequence Diagram

Sequence of operations:

1. Operator logs into system
2. Assembly data is uploaded
3. Entity Recognition model processes data
4. Relation Extraction model identifies relationships
5. Knowledge Graph is updated
6. Robot queries graph for task planning
7. System executes task
8. Feedback and logs are stored

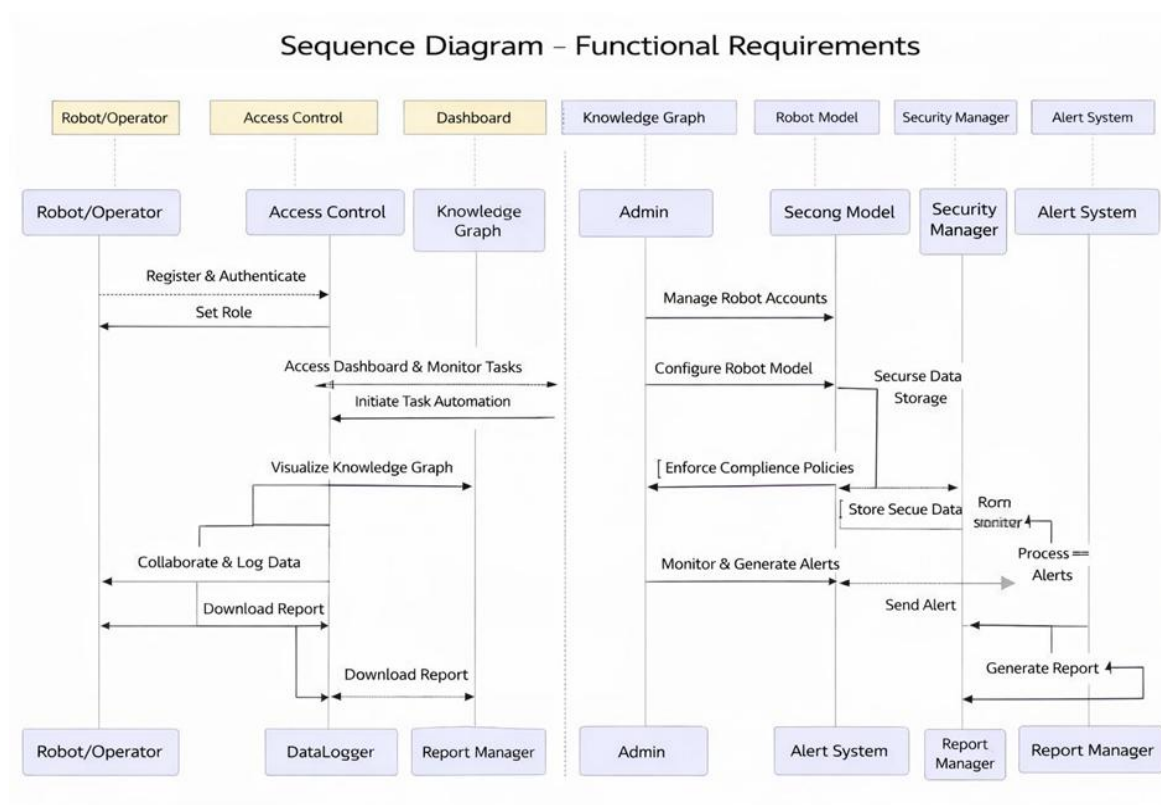


FIG-5.5.3 Sequence Diagram

5.5.4 Activity Diagram

Workflow:

Login → Data Input → Preprocessing → Entity Extraction → Relation Extraction → Knowledge Graph Construction → Decision Making → Task Execution → Monitoring → Feedback Update

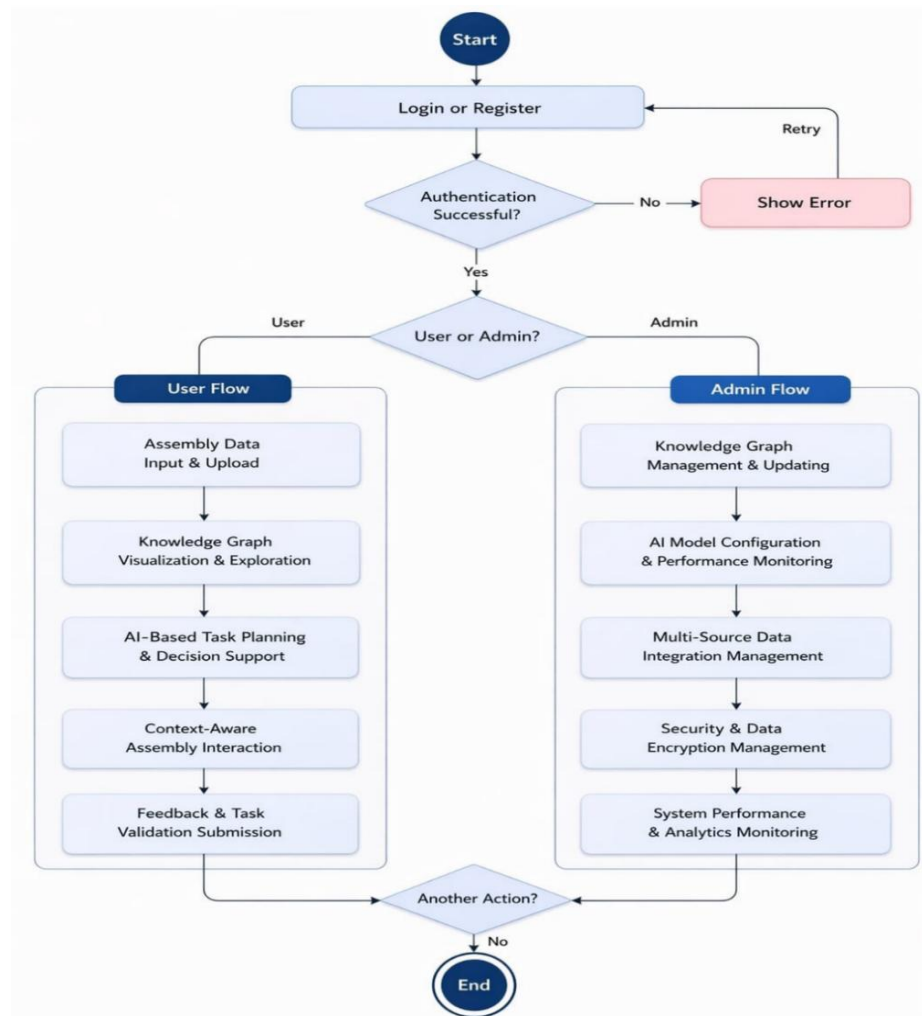


FIG-5.5.4 Activity Diagram

6. IMPLEMENTATION

6.1 Source Code

====Views.py====

```
from django.shortcuts import render, redirect, get_object_or_404

from django.contrib.auth.mixins import LoginRequiredMixin

from django.views.generic import View, TemplateView, ListView

from django.contrib import messages

from django.http import JsonResponse

from django.utils import timezone

from django.db.models import Q

from .models import CollaborationSession, QueryResponse, UserFeedback

from accounts.models import UserActivity

from kg_management.models import KnowledgeGraphNode, KnowledgeGraphRelation

import uuid

import re

class CollaborationView(LoginRequiredMixin, TemplateView):

    template_name = 'collab_support/collaborate.html'

    def get_context_data(self, **kwargs):

        context = super().get_context_data(**kwargs)
```

```

# Get active session for user

active_session = CollaborationSession.objects.filter(

    user=self.request.user,

    is_active=True

).first()

context['active_session'] = active_session

if active_session:

    context['session_queries'] = QueryResponse.objects.filter(

        session=active_session

    ).order_by('timestamp')

return context

class SessionStartView(LoginRequiredMixin, View):

    def post(self, request):

        # End any existing active sessions

        CollaborationSession.objects.filter(

            user=request.user,

            is_active=True

        ).update(is_active=False, ended_at=timezone.now())

        # Create new session

        session = CollaborationSession.objects.create(

```

```

        user=request.user,

        session_id=str(uuid.uuid4()),

        is_active=True

    )

    UserActivity.objects.create(

        user=request.user,

        activity_type='collaboration_started',

        description='Started collaboration session',

        ip_address=request.META.get('REMOTE_ADDR')

    )

    return JsonResponse({

        'status': 'success',

        'session_id': session.session_id,

        'message': 'Collaboration session started'

    })

class SessionEndView(LoginRequiredMixin, View):

    def post(self, request):

        session_id = request.POST.get('session_id')

        if session_id:

            session = get_object_or_404(

```

```

        CollaborationSession,

        session_id=session_id,

        user=request.user

    )

    session.is_active = False

    session.ended_at = timezone.now()

    session.save()

    UserActivity.objects.create(

        user=request.user,

        activity_type='collaboration_ended',

        description='Ended collaboration session',

        ip_address=request.META.get('REMOTE_ADDR')

    )

    return JsonResponse({

        'status': 'success',

        'message': 'Session ended'

    })

    return JsonResponse({

        'status': 'error',

        'message': 'Session not found'

```

```
}, status=404)
```

```
class QueryProcessView(LoginRequiredMixin, View):
```

```
    def post(self, request):
```

```
        query_text = request.POST.get('query', "").strip()
```

```
        session_id = request.POST.get('session_id')
```

```
        if not query_text:
```

```
            return JsonResponse({
```

```
                'status': 'error',
```

```
                'message': 'Query is required'
```

```
            }, status=400)
```

```
        session = get_object_or_404(
```

```
            CollaborationSession,
```

```
            session_id=session_id,
```

```
            user=request.user,
```

```
            is_active=True
```

```
        )
```

```
        # Process query - gets response from Knowledge Graph
```

```
        response_text = self.generate_response(query_text)
```

```
        confidence_score = getattr(self, 'last_confidence', 0.75)
```

```

# Save query and response

query_response = QueryResponse.objects.create(

    session=session,

    query_text=query_text,

    response_text=response_text,

    confidence_score=confidence_score

)

return JsonResponse({

    'status': 'success',

    'response': response_text,

    'confidence': confidence_score,

    'query_id': query_response.id

})

def generate_response(self, query):

    """

    Generate response by searching Knowledge Graph with multiple strategies

    """

    query_lower = query.lower()

    # Strategy 1: Exact keyword matching (highest priority)

    result = self.search_by_exact_keywords(query_lower)

```

```

if result:

    self.last_confidence = result[1]

    return result[0]

# Strategy 2: Category-based search (medium priority)

result = self.search_by_category(query_lower)

if result:

    self.last_confidence = result[1]

    return result[0]

# Strategy 3: Generic knowledge base query (fallback)

response = self.generate_fallback_response(query_lower)

self.last_confidence = 0.65

return response

def search_by_exact_keywords(self, query_lower):

    """Search by exact keywords and patterns"""

    keywords = [w for w in query_lower.split() if len(w) > 2]

    # Try multi-word matches first (higher priority)

    for i in range(len(keywords), 0, -1):

        for j in range(len(keywords) - i + 1):

            keyword_phrase = ''.join(keywords[j:j+i])

            # Try to find exact or partial name matches

```

```

matching_nodes = KnowledgeGraphNode.objects.filter(

    name__icontains=keyword_phrase

)

if matching_nodes.exists():

    node = matching_nodes.first()

    # Determine response type from query intent

    if any(word in query_lower for word in ['available', 'have', 'stock', 'inventory']):

        response = f"<strong>{node.name}</strong> (ID: {node.node_id}) is
available. "

        if node.properties and isinstance(node.properties, dict):

            prop_items = []

            for key, value in list(node.properties.items())[:2]:

                if value and str(value).strip() and key not in ['_id', 'node_id']:

                    prop_items.append(f"{key.replace('_', ' ').title()}: {value}")

            if prop_items:

                response += "Details: " + ", ".join(prop_items) + "."

        return response, 0.90

    elif any(word in query_lower for word in ['what', 'tell', 'info', 'details', 'about']):

        return self._build_detailed_response(node), 0.88

    elif any(word in query_lower for word in ['how', 'procedure', 'steps', 'perform']):

```

```

        return self._build_process_response(node), 0.85

    elif any(word in query_lower for word in ['compatible', 'work with', 'use']):

        return self._build_compatibility_response(node), 0.83

# Fallback: Try single keyword matching

for keyword in keywords[:2]:

    if len(keyword) > 3:

        matching_nodes = KnowledgeGraphNode.objects.filter(

            Q(name__icontains=keyword) |

            Q(properties__icontains=keyword)

        )

        if matching_nodes.exists():

            node = matching_nodes.first()

            if any(word in query_lower for word in ['what', 'tell', 'info']):

                return self._build_detailed_response(node), 0.85

    return None

def _add_properties_to_response(self, node, response):

    """Add node properties to response"""

    if node.properties and isinstance(node.properties, dict):

        prop_items = []

        for key, value in list(node.properties.items())[:3]:

```

```

        if value and str(value).strip():

            prop_items.append(f'{key.replace('_', ' ').title()}: {value}')

    if prop_items:

        response += " Details: " + ", ".join(prop_items) + "."

    return response

def _build_detailed_response(self, node):

    """Build detailed information response"""

    response = f"<strong>{node.name}</strong> (ID: {node.node_id}, Type: {node.node_type})<br>"

    if node.properties and isinstance(node.properties, dict):

        response += "Specifications:<br>"

        for key, value in list(node.properties.items())[:5]:

            if value and str(value).strip():

                response += f"• {key.replace('_', ' ').title()}: {value}<br>"

    # Add relationships

    relations = KnowledgeGraphRelation.objects.filter(

        Q(source_node=node) | Q(target_node=node)

   )[:3]

    if relations.exists():

```

```

response += "Related Items:<br>"

for rel in relations:

    if rel.source_node == node:

        response += f"• {rel.relation_type} → {rel.target_node.name}<br>"

    else:

        response += f"• {rel.source_node.name} → {rel.relation_type} →
{node.name}<br>"

return response

def _build_process_response(self, node):

    """Build process/procedure response"""

    response = f"<strong>Process: {node.name}</strong><br>"

    # Get related items

    relations = KnowledgeGraphRelation.objects.filter(

        Q(source_node=node) | Q(target_node=node)

    )

    if relations.exists():

        response += "Required Components:<br>"

        for rel in relations[:5]:

            if rel.source_node == node:

                response += f"• {rel.target_node.name} ({rel.relation_type})<br>"

```

```

        else:

            response += f"• {rel.source_node.name} ({rel.relation_type})<br>"

    return response

def _build_compatibility_response(self, node):

    """Build compatibility response"""

    response = f"<strong>{node.name}</strong> Compatibility:<br>"

    relations = KnowledgeGraphRelation.objects.filter(

        Q(source_node=node) | Q(target_node=node)

    )

    if relations.exists():

        for rel in relations[:5]:

            if rel.source_node == node:

                response += f"• {rel.relation_type} {rel.target_node.name}<br>"

            else:

                response += f"• {rel.source_node.name} {rel.relation_type} it<br>"

        else:

            response += f"• Compatible with standard assembly components<br>"

            response += f"• Works within our HRC system"

    return response

```

```

def search_by_category(self, query_lower):

    """Search by category keywords"""

    # Statistics first - highest priority

    if any(word in query_lower for word in ['many', 'count', 'total', 'number', 'how many',
'statistics', 'overview', 'capabilities']):

        return self._get_statistics_response(), 0.90

    # Parts/Components

    elif any(word in query_lower for word in ['part', 'component', 'bracket', 'mount',
'bearing', 'plate', 'harness', 'board', 'absorber', 'sensor']):

        return self._get_category_response('part'), 0.85

    # Tools

    elif any(word in query_lower for word in ['tool', 'equipment', 'screwdriver', 'wrench',
'drill', 'gauge', 'press', 'tester', 'pressure']):

        return self._get_category_response('tool'), 0.83

    # Robots

    elif any(word in query_lower for word in ['robot', 'arm', 'automation', 'cobot', 'industrial',
'mobile', 'carrier', 'bot']):

        return self._get_category_response('robot'), 0.8

    # Processes

    elif any(word in query_lower for word in ['process', 'assembly', 'installation', 'procedure',
'workflow', 'mounting', 'wiring', 'calibration']):

        return self._get_category_response('process'), 0.84

```

```

return None

def _get_category_response(self, category):

    """Get response for a specific category"""

    items = KnowledgeGraphNode.objects.filter(node_type=category)

    if not items.exists():

        return f"No {category}s found in the system currently."

    item_list = items[:5]

    response = f"<strong>Available {category.title()}s ({items.count()}
total):</strong><br>"

    for item in item_list:

        response += f"• <strong>{item.name}</strong> (ID: {item.node_id})<br>"

    if items.count() > 5:

        response += f"... and {items.count() - 5} more<br>"

    response += f"<br>Ask for details about any specific {category}!"

    return response

def _get_statistics_response(self):

    """Get system statistics response"""

    parts = KnowledgeGraphNode.objects.filter(node_type='part').count()

    tools = KnowledgeGraphNode.objects.filter(node_type='tool').count()

```

```

robots = KnowledgeGraphNode.objects.filter(node_type='robot').count()

materials = KnowledgeGraphNode.objects.filter(node_type='material').count()

processes = KnowledgeGraphNode.objects.filter(node_type='process').count()

relations = KnowledgeGraphRelation.objects.count()

response = "<strong>Knowledge Base Statistics:</strong><br>"

response += f"• <strong>{parts}</strong> Parts<br>"

response += f"• <strong>{tools}</strong> Tools<br>"

response += f"• <strong>{robots}</strong> Robots<br>"

response += f"• <strong>{materials}</strong> Materials<br>"

response += f"• <strong>{processes}</strong> Processes<br>"

response += f"• <strong>{relations}</strong> Relationships<br>"

response += f"<strong>Total: {parts + tools + robots + materials + processes} entities</strong>"

return response

def generate_fallback_response(self, query_lower):

    """Generate helpful response when no specific match found"""

    parts = KnowledgeGraphNode.objects.filter(node_type='part').count()

    tools = KnowledgeGraphNode.objects.filter(node_type='tool').count()

    robots = KnowledgeGraphNode.objects.filter(node_type='robot').count()

    total = parts + tools + robots

```

```

response = f'I can help with our system! We have:<br>'

response += f'• <strong>{parts}</strong> parts and components<br>'

response += f'• <strong>{tools}</strong> tools and equipment<br>'

response += f'• <strong>{robots}</strong> robots<br><br>'

response += f'<strong>Try asking:</strong><br>'

response += f'• \"What tools do we have?\"<br>'

response += f'• \"Tell me about [item name]\"<br>'

response += f'• \"Is [item] available?\"<br>'

response += f'• \"How compatible are X and Y?\"<br>'

response += f'• \"How many robots do we have?\"<br>'

response += f'<br>Ask about any specific item or process!'

return response

```

```

class FeedbackSubmitView(LoginRequiredMixin, View):

```

```

    def post(self, request):

```

```

        query_response_id = request.POST.get('query_response_id')

```

```

        rating = request.POST.get('rating')

```

```

        comment = request.POST.get('comment', '')

```

```

        is_helpful = request.POST.get('is_helpful', 'true') == 'true'

```

```

        query_response = get_object_or_404(QueryResponse, pk=query_response_id)

```

```

        feedback = UserFeedback.objects.create(

```

```

        user=request.user,

        query_response=query_response,

        rating=int(rating),

        comment=comment,

        is_helpful=is_helpful
    )

    return JsonResponse({

        'status': 'success',

        'message': 'Thank you for your feedback!'

    })

```

```

class CollaborationHistoryView(LoginRequiredMixin, ListView):

```

```

    model = CollaborationSession

```

```

    template_name = 'collab_support/history.html'

```

```

    context_object_name = 'sessions'

```

```

    paginate_by = 10

```

```

    def get_queryset(self):

```

```

        return CollaborationSession.objects.filter(

```

```

            user=self.request.user

```

```

        ).order_by('-started_at')

```

7. SYSTEM TESTING

System testing evaluates whether the AI-Based Human-Robot Collaborative Assembly System works correctly, efficiently and safely as required by the functional specifications. The tests see if the modules worked as they should, and whether the whole system provides intelligent collaborative assembly workflows.

Testing is carried out at different levels such as unit testing, integration testing, system testing, and user acceptance testing etc.

7.1 Testing Objectives

The objectives of system testing are:

- Ensure all functional requirements have been implemented successfully.
- The purpose of these is also to guarantee proper natural language understanding and correct task interpretation
- To ensure the real-time robot responses and task execution can be validated.
- To verify that authentication (to ensure secure authentication) and role-based access control (to ensure role-based access control).
- To test whether the knowledge base is correctly integrated and the learning is capable of reasoning.
- To evaluate the performance of the system in collaborative assembly in real time.
- To provide data integrity, safety compliance, and a safe and secure communication.

7.2 Types of testing

The following types of testing are performed:

7.2.1. Unit Testing

Unit testing is performed on individual modules such as:

- User Authentication Module
- Natural Language Processing (NLP) Module
- Task Planning Module
- Knowledge Base Module
- Robot Control Module
- Monitoring & Dashboard Module

The individual modules are all tested independently in order to ascertain that they function correctly and give the expected output.

Example: Run the NLP module with a variety of assembly instructions to check how well it recognizes entities, detects intent and maps to a task.

7.2.2 Integration Testing

Integration testing-It checks if all the interconnected modules working together without if there is any error.

- User command is implemented by NLP, then sent to the task planning module.
- Task Planning module generates commands and sends these to Robot Control module.
- Knowledge base retrieval in support of live reasoning.
- Assembly execution updates displayed in monitoring dashboard.
- It allows smooth flow of data between entities and proper coordination of the system.

7.2.3 System Testing

System testing examines the fully integrated system to assess for system-wide issues.

It checks:

- Complete workflow from login to complete the tasks.
- Interpreting natural language instructions and executing in robot systems.
- ‘Work’ collaboratively between human and robot in real time.
- Monitoring and recording of task progress.
- Error detection and recovery.
- The platform is validated under simulated industrial assembly conditions.

7.2.4 User Acceptance Testing (UAT)

User Acceptance Testing is performed with operators and engineers to confirm real-world usability and cooperative work efficiency.

The following aspects are evaluated:

- Correctness in execution of the instructions given.
- Response delay of robot
- Simple to interact with the system
- Dashboard clarity 4.1 The dashboard should be clear and easy for a child to understand. Features requiring explanation are signs of too high a level of complexity.
- Safety adhering to when collaboration
- Comments are received for further refining, enhancing, and usability of the system.

7.2.5 Performance Testing

Performance testing can guarantee the system works efficiently in real time assembling process.

The system is tested for:

- Response time of language processing
- Robot command execution delay
- Multiple tasks at the same time2.4.74.
- Server load handling during operation
- How about scalability for industrial environments
- It should be able to understand commands and carry out a robotic task in a matter of milliseconds so that collaboration is efficient.

7.2.6 Security Testing

Security testing is a verification that the information is protected from sensitive operational and system data.

Security checks include:

- Password encryption validation
- Role-based access control Testing
- Detection of tags making unauthorised access
- Validation of secure robot communication protocol
- Audit log and activity monitoring.
- Protects against unauthorized access and ensures safe collaborative use.

7.2.7 Usability Testing

Usability is the easy of use of interface. How easy it is for the operator to interact with the interface.

It ensures:

- Intuitive command input interface
- Intuitive monitoring dashboard:
- Simple task creation and editing
- Robot status can be presented on a clear visual way
- Operator training needs: operators do not need to be skilled.
- The interface just has to serve the needs of the industrial engineers and the technicians.

It is not as sophisticated or complex as possible.

7.3 Test Cases

Test Case ID	Test Case Name	Module	Input	Expected Output	Actual Output	Status
TC_01	Valid User Login	Authentication Module	Correct username & password	User successfully logged in and dashboard displayed	As Expected	Pass
TC_02	Invalid User Login	Authentication Module	Incorrect password	Error message displayed: "Invalid Credentials"	As Expected	Pass
TC_03	Natural Language Instruction Processing	NLP Module	"Attach the gear to the motor shaft"	Entities and task correctly extracted	As Expected	Pass
TC_04	Task Planning Generation	Task Planning Module	Recognized task data	Valid robot action sequence generated	As Expected	Pass
TC_05	Robot Command Execution	Robot Control Module	Generated action sequence	Robot performs assembly step correctly	As Expected	Pass
TC_06	Knowledge Base Retrieval	Knowledge Module	Complex assembly query	Relevant domain knowledge retrieved	As Expected	Pass

Test Case ID	Test Case Name	Module	Input	Expected Output	Actual Output	Status
TC_07	Dashboard Status Update	Monitoring Module	Task completion signal	Assembly progress updated in dashboard	As Expected	Pass
TC_08	Multi-Step Instruction Handling	NLP + Planning	Sequential assembly instructions	Correct multi-step execution	As Expected	Pass
TC_09	Error Handling Mechanism	Control Module	Invalid assembly command	Error message and safe stop triggered	As Expected	Pass
TC_10	Unauthorized Access Attempt	Security Module	Direct system access without login	Redirected to login page	As Expected	Pass

Table 7.3.1 Test Cases

7.4 Discussion of Results

All modules were tested successfully under controlled industrial simulation conditions.

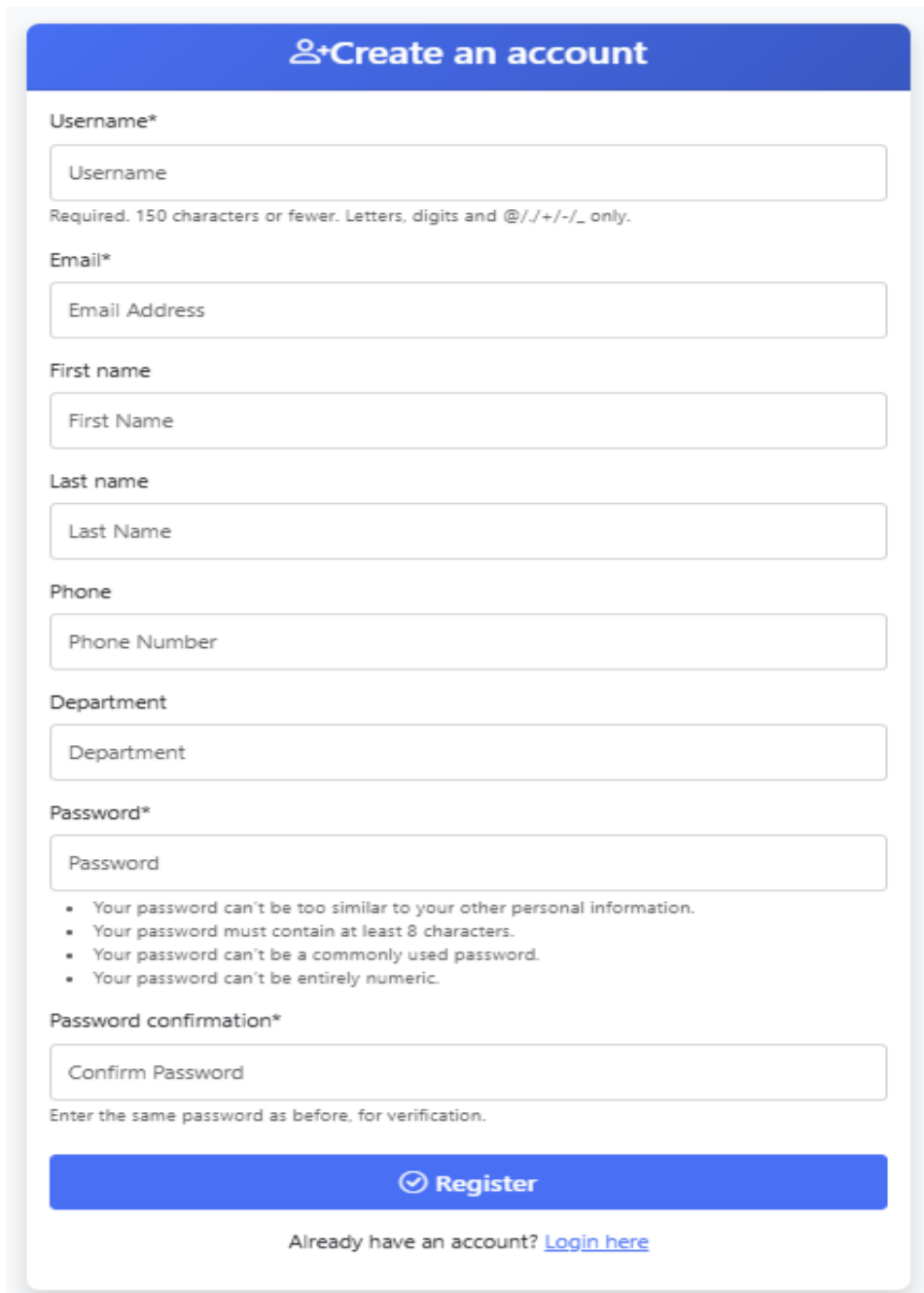
- Functional modules performed as expected.
- NLP accurately interpreted assembly instructions.
- Knowledge-based reasoning enhanced task understanding.
- Security mechanisms prevented unauthorized access.
- System performance remained stable under moderate operational load.

The AI-Based Human-Robot Collaborative Assembly System demonstrated reliable performance, accurate reasoning capability, and effective real-time collaborate

8. SCREENSHOTS

REGISTRATION FORM

- A registration module that allows new users to create an account by entering personal and system-related details.
- The collected data is validated and stored securely for future authentication and access



The screenshot shows a registration form with a blue header bar containing a user icon and the text "Create an account". The form fields are as follows:

- Username***: A text input field with placeholder text "Username". Below it, a note states: "Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only."
- Email***: A text input field with placeholder text "Email Address".
- First name**: A text input field with placeholder text "First Name".
- Last name**: A text input field with placeholder text "Last Name".
- Phone**: A text input field with placeholder text "Phone Number".
- Department**: A text input field with placeholder text "Department".
- Password***: A text input field with placeholder text "Password". Below it, a list of password requirements is shown:
 - Your password can't be too similar to your other personal information.
 - Your password must contain at least 8 characters.
 - Your password can't be a commonly used password.
 - Your password can't be entirely numeric.
- Password confirmation***: A text input field with placeholder text "Confirm Password". Below it, a note states: "Enter the same password as before, for verification."

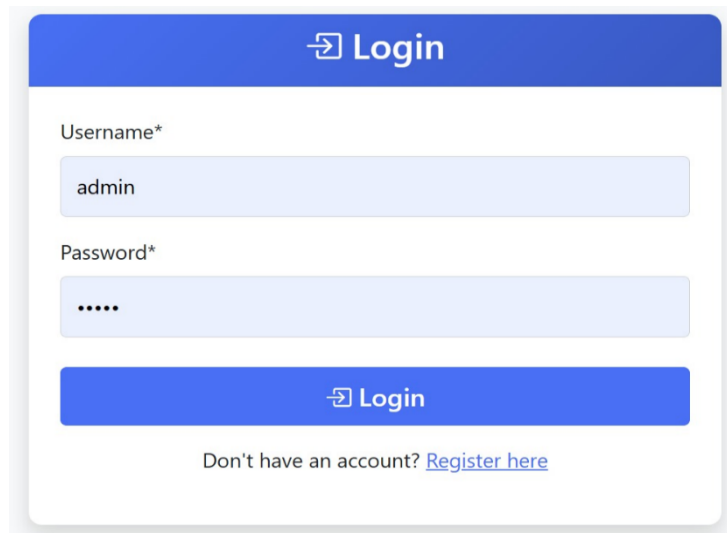
At the bottom of the form is a blue button with a checkmark icon and the text "Register". Below the button is a link: "Already have an account? [Login here](#)".

Screenshot 8.1 Registration Page

LOGIN PAGE

ADMIN LOGIN

- A secure login portal for administrators to manage system operations and monitor knowledge graph construction.
- Admins can control datasets, oversee deep learning processes, and maintain system integrity.

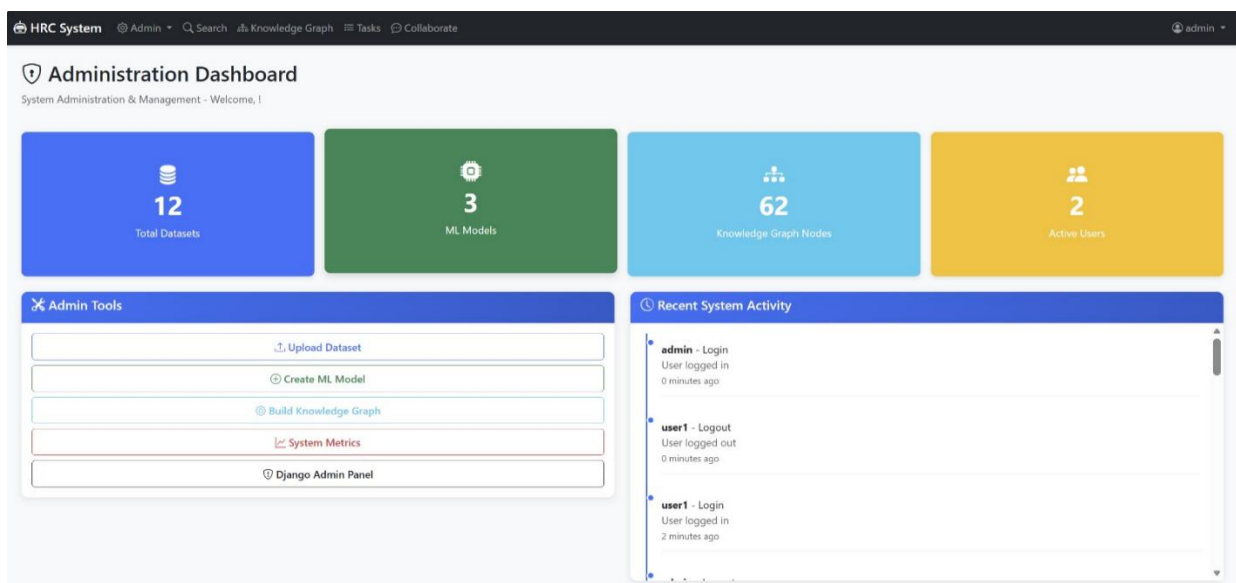


A screenshot of the Admin Login page. It features a blue header with a white 'Login' button and a white icon. Below the header, there are two input fields: 'Username*' with the text 'admin' and 'Password*' with masked characters '.....'. A blue 'Login' button with a white icon is positioned below the password field. At the bottom, there is a link that says 'Don't have an account? [Register here](#)'.

Screenshot 8.2 Admin Login

ADMIN DASHBOARD

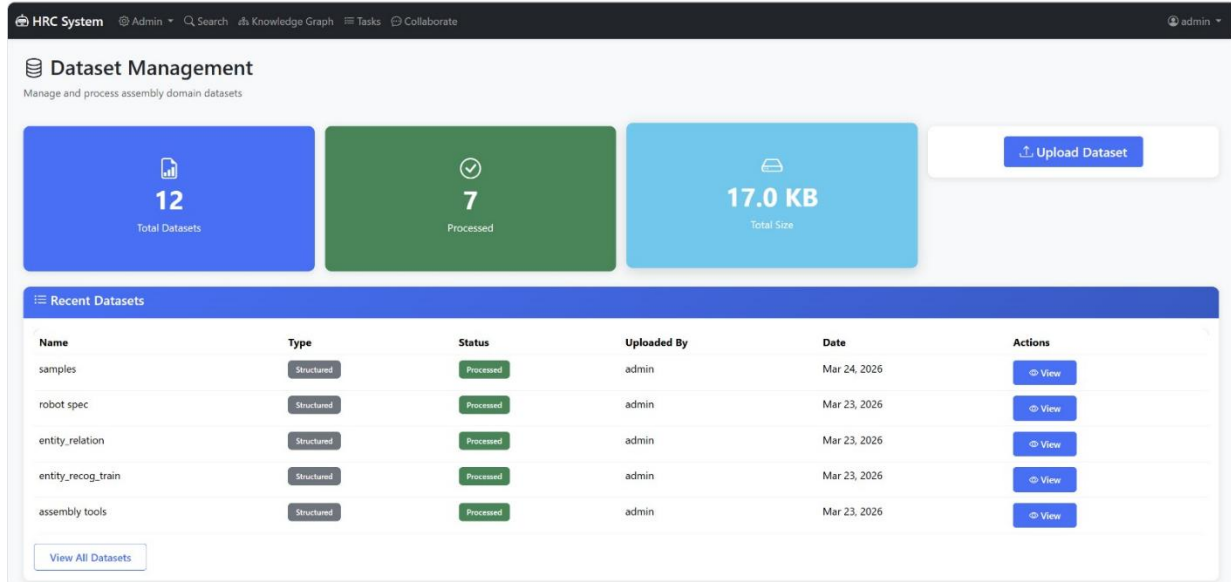
- A centralized control panel for administrators to monitor system activities and manage knowledge graph construction processes.
- It provides access to data handling, model execution, and overall system management features.



Screenshot 8.3 Admin Dashboard

DATA SET MANAGEMENT

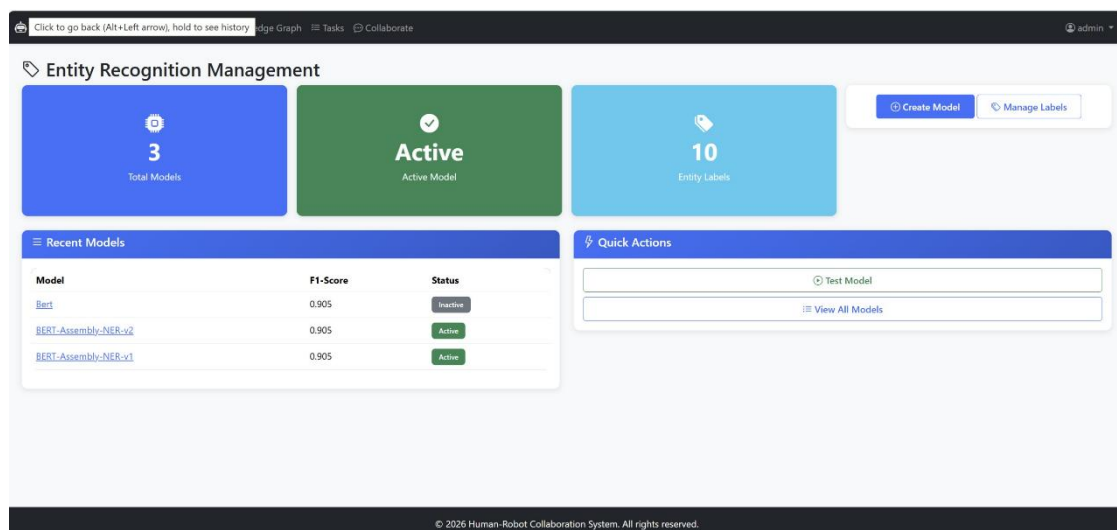
- A module that allows admins to upload, update, and organize datasets used for training deep learning models.
- Ensures data quality and proper structuring for effective knowledge graph generation.



Screenshot 8.4 Dataset Management

ENTITY RECOGNITION MANAGEMENT

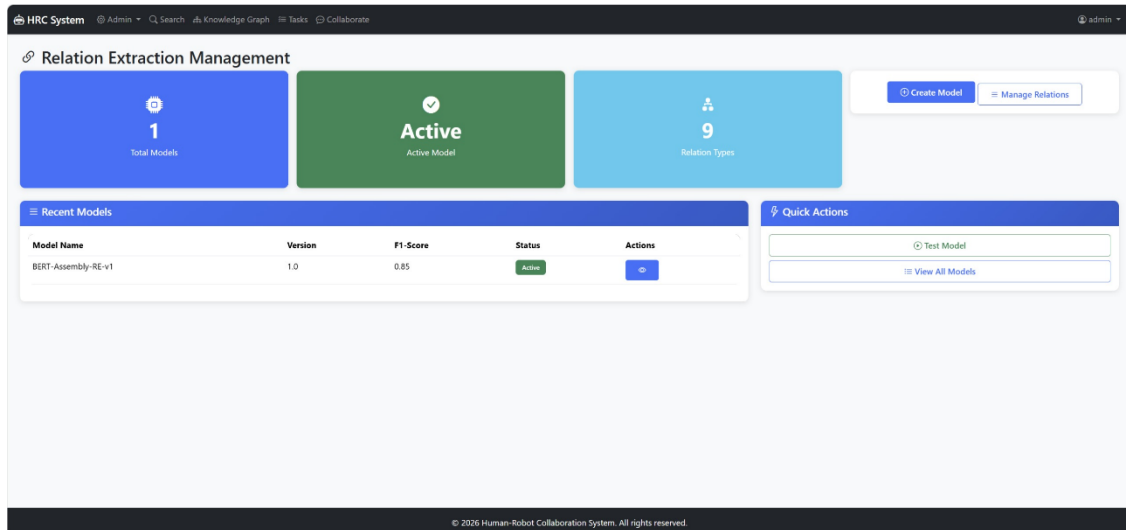
- A module for managing and configuring entity recognition processes used in knowledge graph construction.
- It enables handling of extracted entities and ensures accurate identification from assembly domain data.



Screenshot 8.5 Entity Recognition Management

RELATION EXTRACTION MANAGEMENT

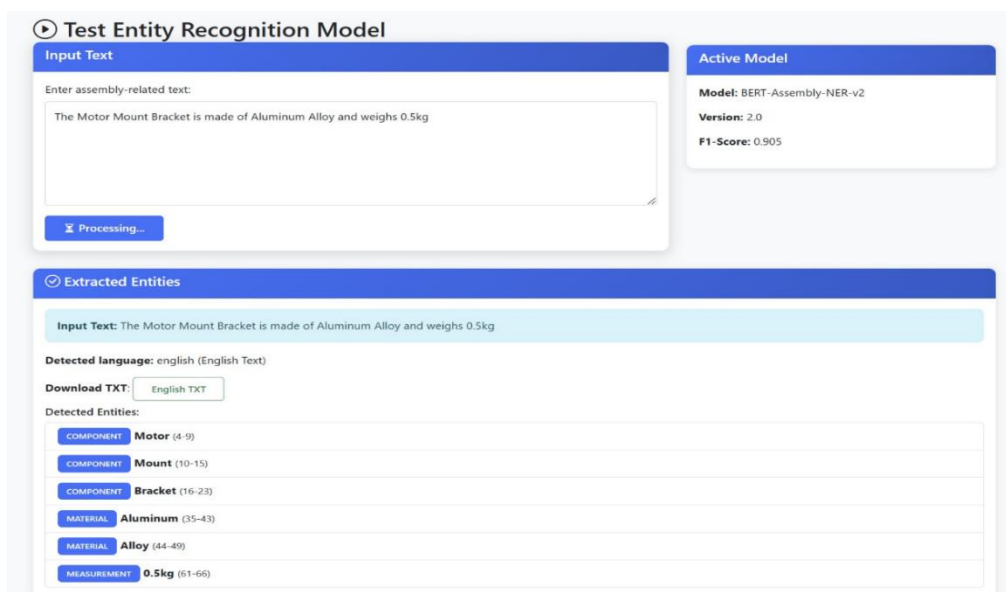
- A module for managing the extraction of relationships between identified entities within the dataset.
- It ensures accurate linking of entities to form meaningful connections in the knowledge graph.



Screenshot 8.6 Relation Extraction Management

TEST ENTITY RECOGNITION MODEL

- A feature to evaluate the performance of the trained entity recognition model on test data.
- Verifies accuracy and effectiveness before integrating results into the knowledge graph.



Screenshot 8.7 Test Entity Recognition Model

TEST ENTITY RECOGNITION MODEL (CHINESE)

- A module used to evaluate the entity recognition model on Chinese text data
- Ensures accurate identification of entities specific to the Chinese assembly domain

The screenshot shows a web interface for testing an entity recognition model on Chinese text. The title is "Test Entity Recognition Model". It has two main sections: "Input Text" and "Active Model".

Input Text: A text area contains the Chinese sentence: "电机安装支架由铝合金制成, 重量为 0.5 公斤。这是装配操作中的关键部件。". Below the text area is a "Processing..." button.

Active Model: A box on the right shows the model details: "Model: BERT-Assembly-NER-v2", "Version: 2.0", and "F1-Score: 0.905".

Extracted Entities: A section below the input text shows the detected language as "chinese (Chinese Text)". It includes a "Download TXT" button labeled "Chinese TXT". Below this, a table lists the detected entities:

Entity Type	Entity Name	Start Index	End Index
COMPONENT	电机	0	2
COMPONENT	支架	4	6
MATERIAL	铝	7	8
MATERIAL	合金	8	10
PROPERTY	重量	13	15
MEASUREMENT	0.5 公斤	17	23

Screenshot 8.8 Test Entity Recognition Model (Chinese)

TEXT RELATION EXTRACTION MODEL

- A model that identifies and extracts relationships between entities from textual data
- Helps build meaningful connections required for knowledge graph construction

The screenshot shows a web interface for testing a text relation extraction model. The title is "Test Relation Extraction Model". It has two main sections: "Input" and "Active Model".

Input: A text area contains the English sentence: "The Motor Mount Bracket is made of Aluminum Alloy". Below the text area are two input fields for "Entity 1" (Motor Mount Bracket) and "Entity 2" (Aluminum Alloy). A "Processing..." button is located below these fields.

Active Model: A box on the right shows the model details: "Model: BERT-Assembly-RE-v1", "Version: 1.0", and "F1-Score: 0.85".

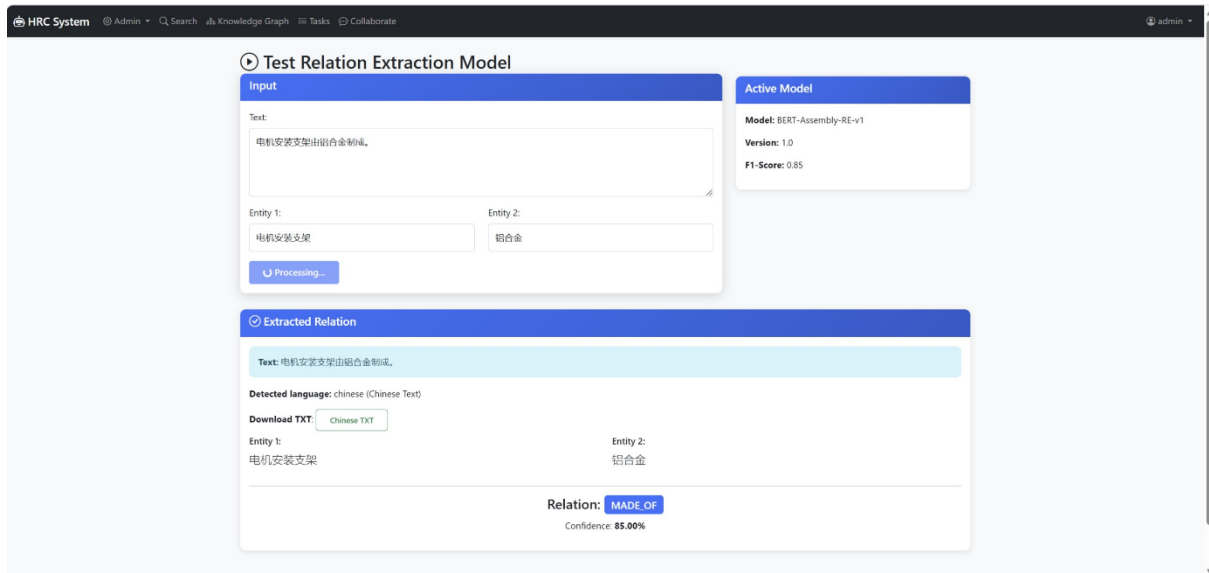
Extracted Relation: A section below the input text shows the detected language as "english (English Text)". It includes a "Download TXT" button labeled "English TXT". Below this, a table lists the extracted relation:

Text	Entity 1	Entity 2	Relation	Confidence
The Motor Mount Bracket is made of Aluminum Alloy	Motor Mount Bracket	Aluminum Alloy	MADE_OF	85.00%

Screenshot 8.9 Test Relation Extraction Model

TEXT RELATION EXTRACTION MODEL (CHINESE)

- Improves knowledge graph quality for Chinese assembly domain content
- Enables consistent and reliable relation extraction across languages

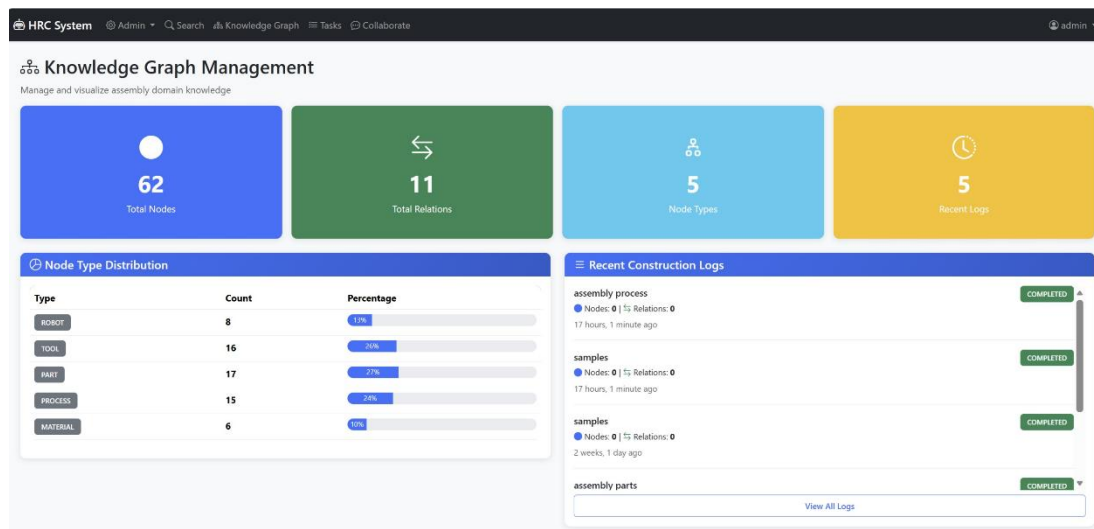


Screenshot 8.10 Test Relation Extraction Model (Chinese)

KNOWLEDGE GRAPH

MANAGEMENT DASHBOARD

- A module that manages the creation, updating, and visualization of the knowledge graph structure.
- It allows organizing entities and relationships to build a structured representation of assembly domain knowledge.



Screenshot 8.11 Knowledge Graph Management

CREATE KNOWLEDGE GRAPH NODE

- A feature that enables adding new nodes representing entities into the knowledge graph.
- Each node stores relevant information to support connections and graph-based analysis.

Create Knowledge Graph Node

Node id*
Y0011

Node type*
Part

Name*
Motor Mount Bracket

Properties

```
{  
  "part_type": "Bracket",  
  "material": "Aluminum Alloy",  
  "weight_kg": 0.5,  
  "dimensions": "100x80x20mm",  
  "supplier": "ABC Industries",  
}
```

Enter properties as JSON

Create Node Cancel

Screenshot 8.12 Create Knowledge Graph Node

REPRESENTATION OF NODE

- Represents a specific assembly component used as an example entity within the knowledge graph.
- It helps demonstrate how real-world parts are modelled and connected in the assembly domain.

KG Management / Nodes / Motor Mount Bracket

Motor Mount Bracket

Node ID: Y0011

Type: PART

Name: Motor Mount Bracket

Properties

Property	Value
cost_inr	450
material	Aluminum Alloy
supplier	ABC Industries
part_type	Bracket
weight_kg	0.5
dimensions	100x80x20mm

Created By: admin

Created At: April 09, 2026 11:02:45

Last Updated: April 09, 2026 11:02:45

Relationships

Outgoing Relationships
No outgoing relationships

Incoming Relationships
No incoming relationships

Actions

Edit Node

Visualize in Graph

Back to List

Delete Node

Statistics

Total Relations: 0

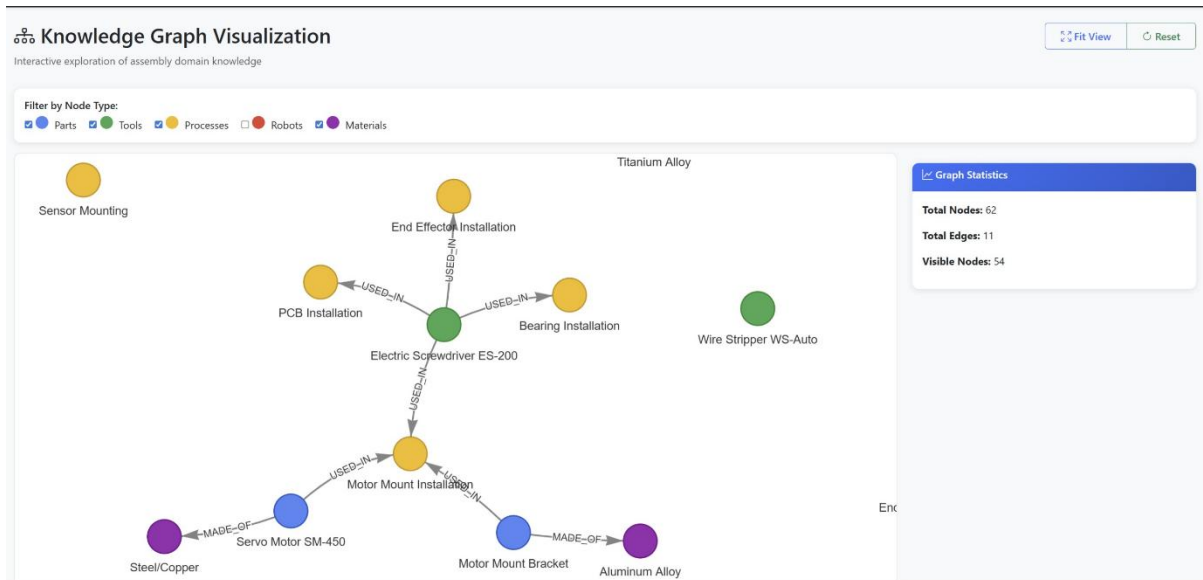
Outgoing: 0

Incoming: 0

Screenshot 8.13 Representation of Node

KNOWLEDGE GRAPH VISUALIZATION

- A feature that displays the constructed knowledge graph for easy understanding.
- It allows users to explore entities and their relationships interactively.



Screenshot 8.14 Knowledge Graph Visualization

MY ASSEMBLY TASKS

- A dashboard that displays all assembly tasks created by the user in an organized manner
- Allows users to track progress, view task details, and manage ongoing activities
- Provides quick access to task-related operations and updates
- Enhances workflow efficiency through centralized task monitoring

HRC System | Admin | Search | Knowledge Graph | Tasks | Collaborate | admin

My Assembly Tasks | Create Task

Summary: 1 Pending, 1 In Progress, 2 Completed, 0 Failed

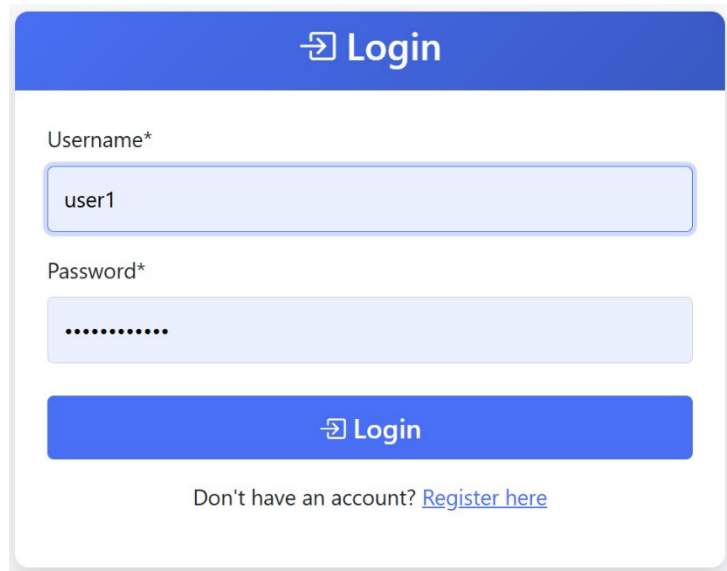
Task Name	Priority	Status	Progress	Created	Actions
task1	Priority 1	Completed		Apr 09, 2026	View
task1	Priority 1	Completed	100%	Mar 23, 2026	View
Gear Assembly and Torque Check	Priority 3	Pending		Mar 23, 2026	View
Motor Assembly Line 01	Priority 2	In Progress		Mar 23, 2026	View

© 2026 Human-Robot Collaboration System. All rights reserved.

Screenshot 8.15 My Assembly Tasks

USER LOGIN

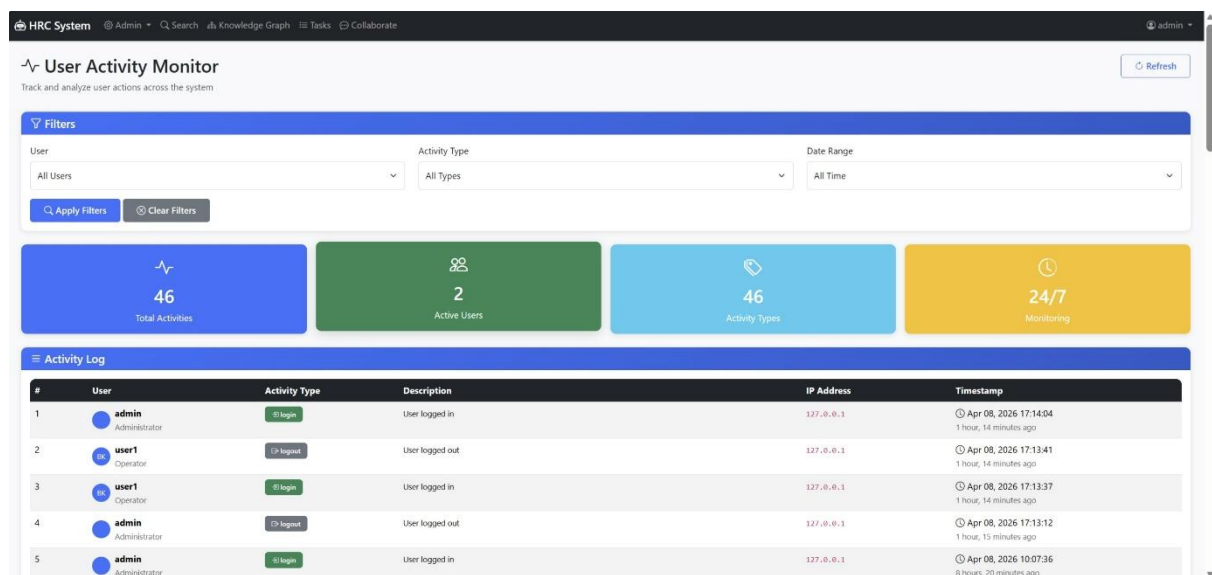
- A login interface where users can enter their credentials to access the knowledge graph system.
- Based on authentication, users are redirected to features like data exploration and graph visualization.



Screenshot 8.16 User login

USER ACTIVITY MONITOR

- A module that tracks and records user actions within the system
- Helps analyze user behavior and interaction patterns
- Provides logs for auditing and security purposes
- Ensures transparency and proper usage of system features



Filters

User: All Users | Activity Type: All Types | Date Range: All Time

Summary Cards:

- 46 Total Activities
- 2 Active Users
- 46 Activity Types
- 24/7 Monitoring

Activity Log

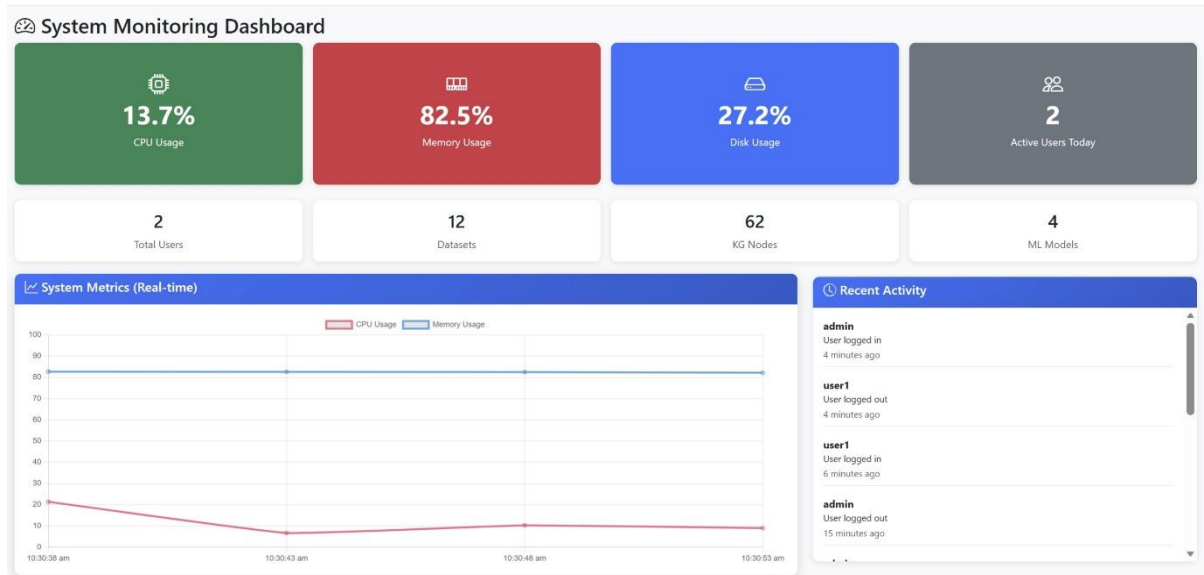
#	User	Activity Type	Description	IP Address	Timestamp
1	admin Administrator	login	User logged in	127.0.0.1	Apr 08, 2026 17:14:04 1 hour, 14 minutes ago
2	user1 Operator	logout	User logged out	127.0.0.1	Apr 08, 2026 17:13:41 1 hour, 14 minutes ago
3	user1 Operator	login	User logged in	127.0.0.1	Apr 08, 2026 17:13:37 1 hour, 14 minutes ago
4	admin Administrator	logout	User logged out	127.0.0.1	Apr 08, 2026 17:13:12 1 hour, 15 minutes ago
5	admin Administrator	login	User logged in	127.0.0.1	Apr 08, 2026 10:07:36 8 hours, 20 minutes ago

Screenshot 8.17 User Activity Monitor

SYSTEM MONITORING DASHBOARD

- A dashboard that displays real-time system performance and operational status
- Monitors resource usage, processes, and overall system health

Helps detect issues and ensure smooth functioning of the application



Screenshot 8.18 System Monitoring Dashboard

9. CONCLUSION AND FUTURE SCOPE

9.1 CONCLUSION

The AI-Based Human-Robot Collaborative Assembly System is an intelligent system of intelligent human-robot interaction in complex assembly environments. Conventional robotic assembly systems are based on programmed instructions and pre-determined task flows. The innovative approach of the system is to employ combined technologies of NLP, knowledge based reasoning, task planning algorithms, and real-time robotic control framework.

Natural language commands can be made by human operators for assembly instructions that are handled by more advanced entity extraction and intent recognition mechanisms. Using context reasoning and factoring domain knowledge, the limitations within rule-based systems are tackled to achieve better understanding of assembly operations. And then task planning module is guided to derive robotic command that would be understood and executed accordingly.

All the system has been tested by detailed testing such as unity testing, integration testing, performance and security test, the system was steady and correct to transfer, execute instruction and perform task. And the dashboard monitor make the task progress, error finding and operation clear for engineer and technician.

Additionally, the use of authentication methods in the secure way, encrypted data transfer and Role Based Access Control can guarantee the integrity of data and safety in operation. Performance testing proved that the entire system is real-time, which is very significant in collaborative work industrial environment.

The implementation of the proposed system enhances three key aspects that are critical to human robot collaborative assembly, flexibility (parameterized nature of system), reasoning ability (omission and addition of the task to be performed) and efficiency (automatic, autonomous and complete operation). In general, the above system can fulfill the industry requirements in terms of automation, optimizations and eventually increasing the production productivity. The manual presence in the system can be minimized and the information flow can be increased thus leading to a faster system. The architecture can be scalable and being driven to the future trends of “smart manufacturing” under Industry 4.0.

9.2 FUTURE SCOPE

Despite the intelligent cooperation between human and robot achieved by the proposed system, some improvement need to be done to enhance the system power and its industrial application can be foreseen.

- Initially, it is beneficial to use state-of-art deep learning model such as language model based on transformer for better context understanding and processing multiple language instruction. Robots will be able to understand more complex conversational instruction at much higher success rate.
- Another approach that would enable the robots to precisely identify the components used in the assembly, and minimize reliance on pre-programmed, structured inputs and instructions, is the implementation of computer vision technology utilizing feedback, like the use of real-time object detection coupled with 3D pose estimation.
- Third, reinforcement learning methods may be incorporated so that robots learn from humans and their previous assembly experiences, eventually autonomously performing task planning, error correction by themselves.
- Fourth, cloud-edge collaborative architecture can also be adopted for scalable architecture and high computational efficiency. Robotics control would be provided by edge-computing, while cloud-based AI service offered expert knowledge for multiple robotic systems.
- Improved safety mechanisms, such as predictive risk detection and human motion tracking can be implemented to provide better collaborative safety standards, without violating any industrial safety rules involving human-robot cooperation.

Moreover, the Digital Twin technology can get a simulation of work flows and processes before implementation which avoids the errors and time consuming operations. In future version, the system can be extended to multi-robot collaboration, which allows multi-robot cooperation for assembly tasks in large manufacturing plants.

These improvements will boost the system to be an autonomous, intelligent, and collaborative solution with strong adaptiveness for future smart factories.

REFERENCES

[1] M. A. Goodrich and A. C. Schultz, “Human–robot interaction: A survey,” *Foundations and Trends in Human–Computer Interaction*, vol. 1, no. 3, pp. 203–275, 2007.

<https://www.nowpublishers.com/article/Details/HCI-013>

[2] O. Khatib, “Real-time obstacle avoidance for manipulators and mobile robots,” *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.

<https://journals.sagepub.com/doi/10.1177/027836498600500106>

[3] T. Winograd, “Understanding natural language,” *Cognitive Psychology*, vol. 3, no. 1, pp. 1–191, 1972.

<https://www.sciencedirect.com/science/article/pii/0010028572900019>

[4] J. Devlin et al., “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019.

<https://arxiv.org/abs/1810.04805>

[5] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, pp. 436–444, 2015.

<https://www.nature.com/articles/nature14539>

[6] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

<http://incompleteideas.net/book/the-book-2nd.html>

[7] A. Bauer et al., “Human-robot collaboration: A survey,” *International Journal of Humanoid Robotics*, vol. 5, no. 1, pp. 47–66, 2008.

<https://www.worldscientific.com/doi/10.1142/S0219843608001303>

[8] ISO 10218-1, “Robots and robotic devices—Safety requirements for industrial robots,” International Organization for Standardization, 2011.

<https://www.iso.org/standard/51330.html>

[9] T. B. Sheridan, “Human–robot interaction: Status and challenges,” *Human Factors*, vol. 58, no. 4, pp. 525–532, 2016.

<https://journals.sagepub.com/doi/10.1177/0018720816644364>